

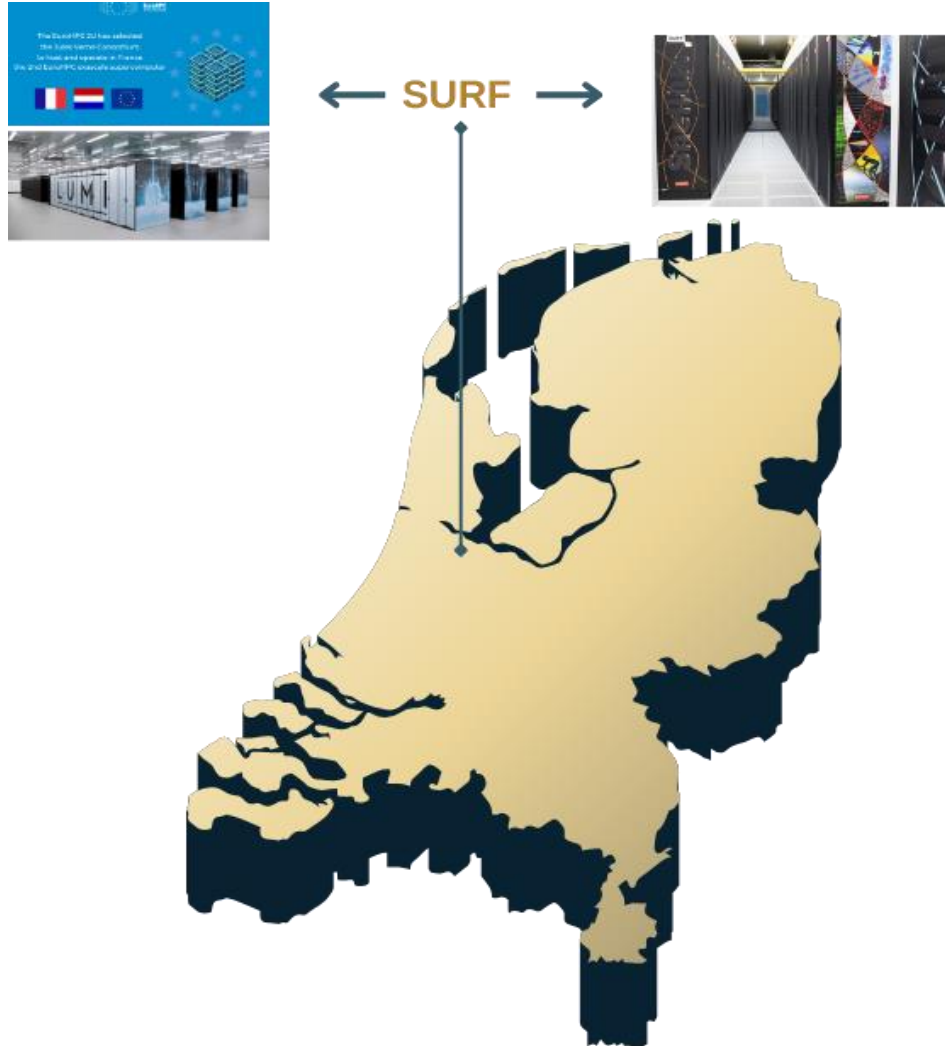


GPU computing with Python

Clícia Pinto
HPC Advisor, SURF

October 7th

Who are we: SURF HPC Advisors



SURF Expertise Pool:

- High-Performance Computing
- High-Performance Machine Learning
- High-performance visualization
- Quantum Computing
- Data preservation
- Data management
- Cloud for Research
- System administration
- Trust and Identity
- Software Management

Who are we: SURF HPC Advisors



Amsterdam Science Park

Outline

- You will write, validate, and measure GPU kernels
- Explain how CUDA maps work to threads, blocks, and warps
- Implement array operations with PyCUDA and Numba-CUDA
- Use shared memory safely when threads reuse or rearrange data
- Compare kernel time with application time and explain the difference

Time	Topic
09:30–10:00	Intro, Python tools, CPU and GPU architecture
10:00–10:20	CUDA memory model and usability
10:20–10:40	Pycuda and numba Exemple #1: vector addition and timing (pycuda)
10:40–10:50	Short Break
10:50–11:30	Cuda Programming Model Exemple #2: Matrix addition and timing (pycuda) Exemple #3: Vector addition and timing (numba)
11:30–12:15	Keynote
12:15–13:00	Lunch
13:00–13:30	Cuda Programming Model Exemple #4: Matrix multiplication and timing
13:30–14:00	Cuda Programming Model - Shared memory Exemple #5: Parallel Reduction
14:00–14:45	Coffee break
14:45–15:00	Short Break
15:00–15:30	Performance and profiling diagnosis
15:30–16:00	Review and questions

Resources

- Slides and source code of the examples can be found at:
 - <https://github.com/sara-nl/Parallel-and-GPU-programming-in-Python/>
- Programming environment (jupyter or vscode):
 - <https://ondemand.snellius.surf.nl/>
- To reset your password and Accept the User Agreement:
 - <https://portal.cua.surf.nl>
- To have more information on Snellius partitions and hardware:
 - <https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/30660209/Snellius+partitions+and+accounting>

Environment OOD Jupyter (VSCode also possible)

1. Login to OOD: <https://ondemand.snellius.surf.nl/>

2. Open the shell

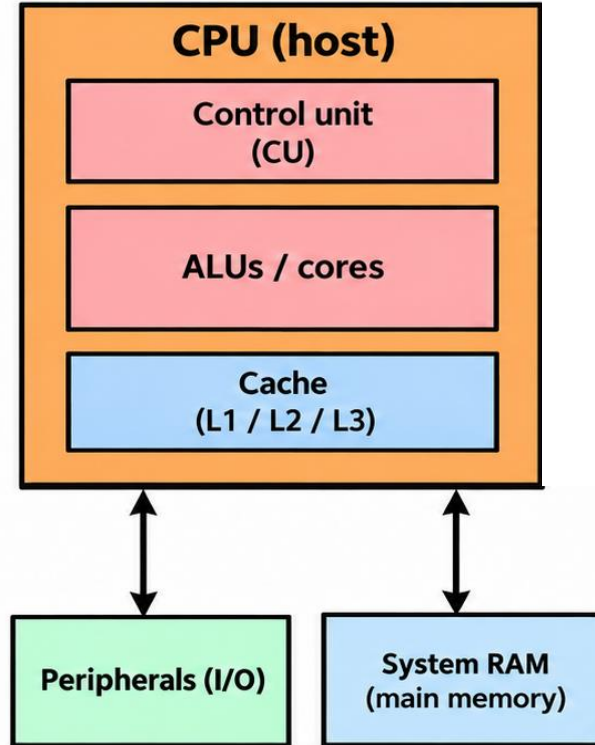
3. Install packages (If needed):

- *\$ module load 2024*
- *\$ module load CUDA/12.6.0*
- *\$ module load Python/3.12.3-GCCcore-13.3.0*

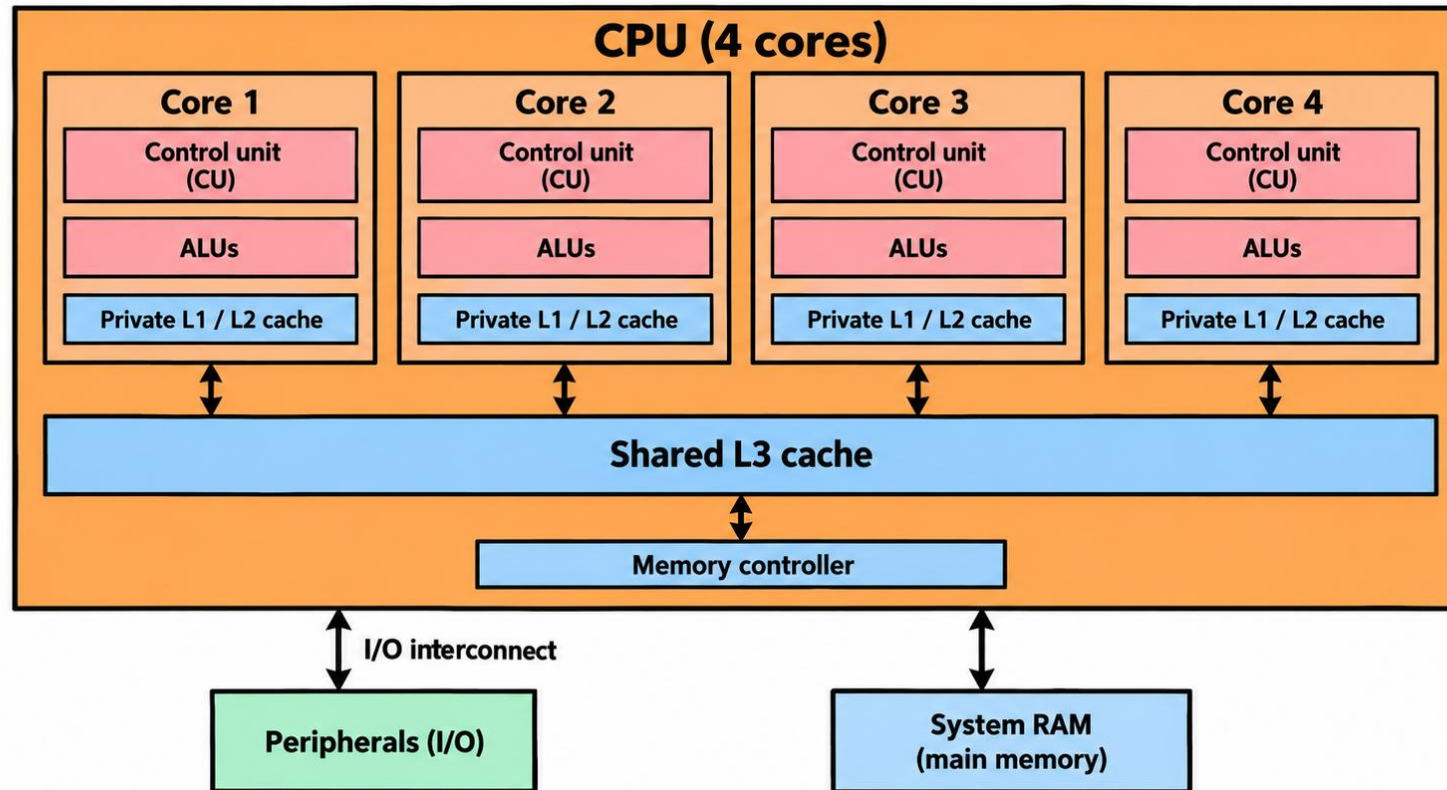
- *\$ pip install pycuda*
- *\$ pip install numba*

- *\$ mkdir gpucourse*
- *\$ cd gpu course*
- *\$ wget <https://github.com/sara-nl/Parallel-and-GPU-programming-in-Python/archive/refs/heads/main.zip>*
- *\$ unzip main*

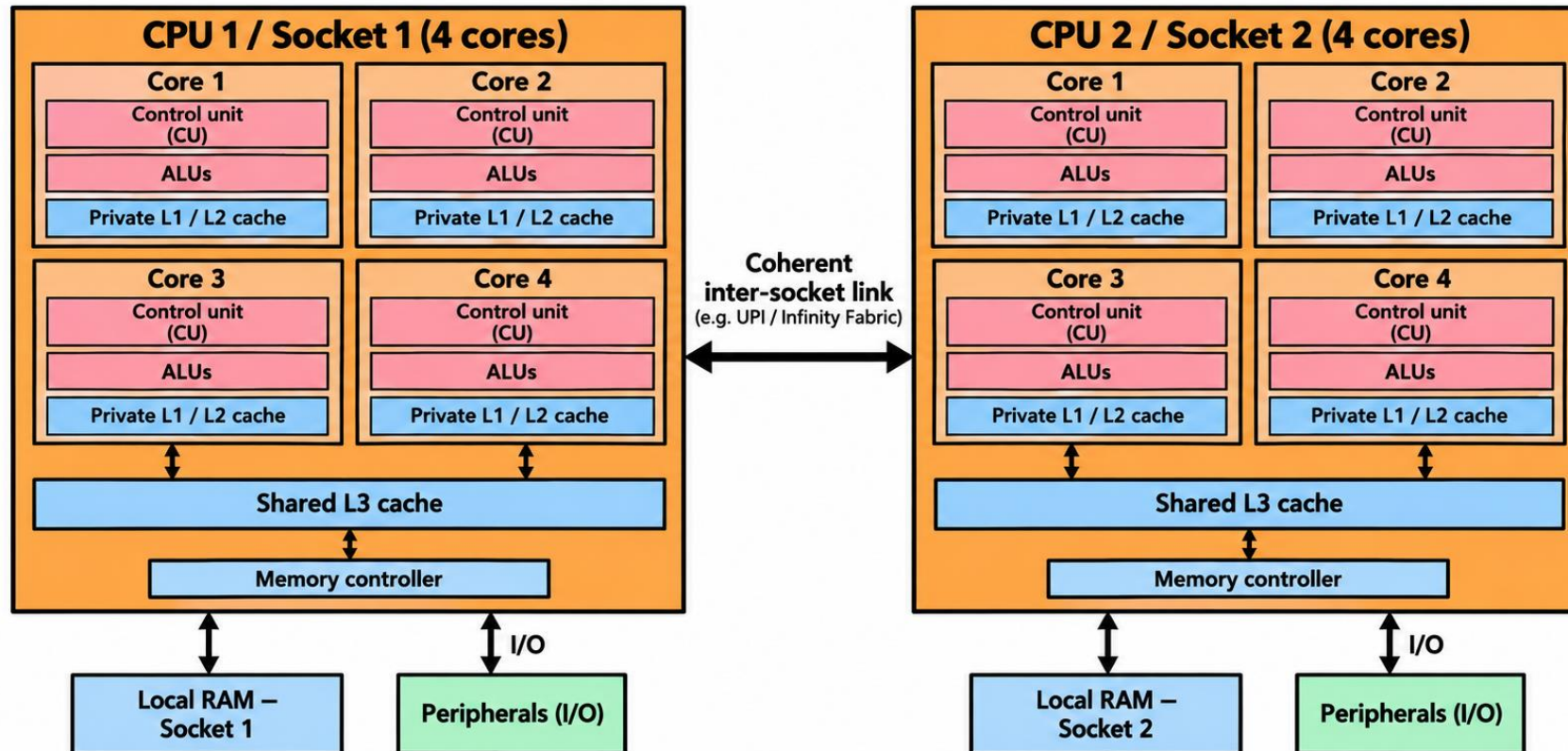
CPU Architecture: What a computer is



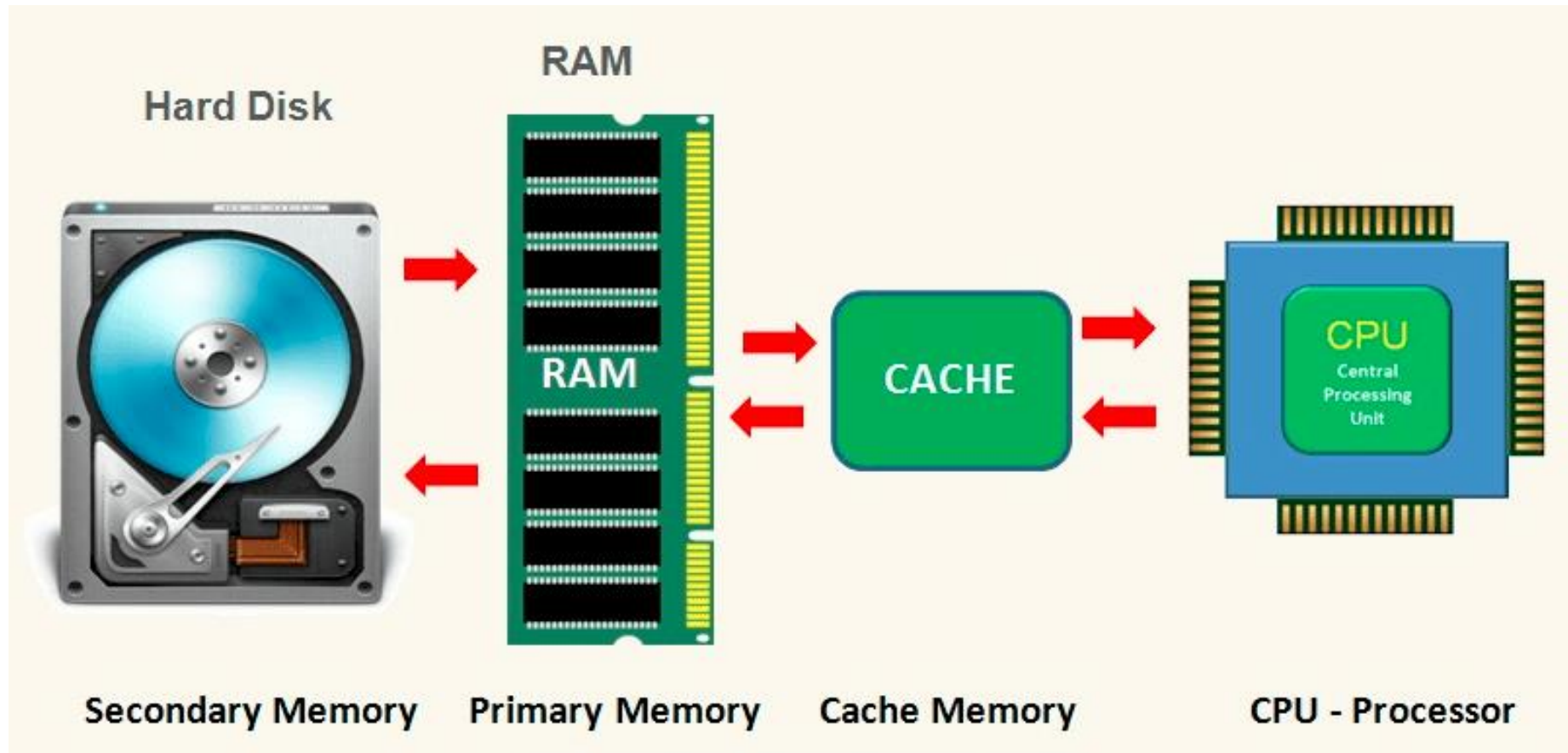
CPU Multicore



CPU Multiprocessor



Memory hierarchy and performance



Parallel and distributed computing

- Larger simulations and datasets require more processing units (workers) and more memory.
- Power and heat limit how much faster a single processor can run.
- We gain capacity by sharing the work across more computing resources.

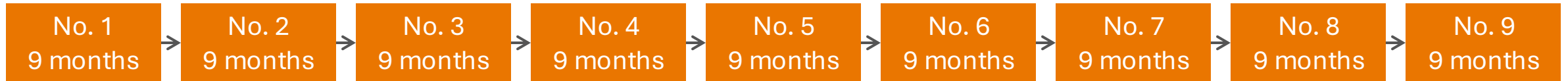
Sequential	Parallel: within one computer	Distributed: across computers
One worker completes tasks one after another.	Several CPU cores or GPU threads work at the same time.	Several computers share the workload through a network.
Example: process images one by one.	Example: different cores process different images.	Example: different computers process different image batches.
Uses one worker.	Uses more execution units to reduce elapsed time when work can overlap.	Adds computing power and memory beyond one machine.

The work must be divisible. Communication and coordination take time.

Parallelism increases output, not every task's speed

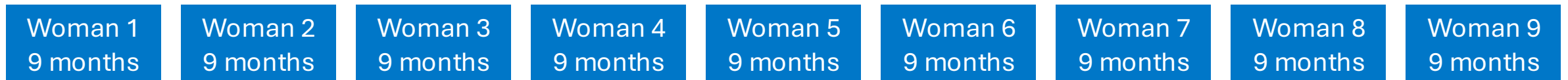
Illustrative model: each pregnancy takes 9 months and results in one baby.

Sequential: one woman, nine pregnancies in sequence



9 pregnancies × 9 months = 81 months

Parallel: nine women, nine pregnancies at the same time



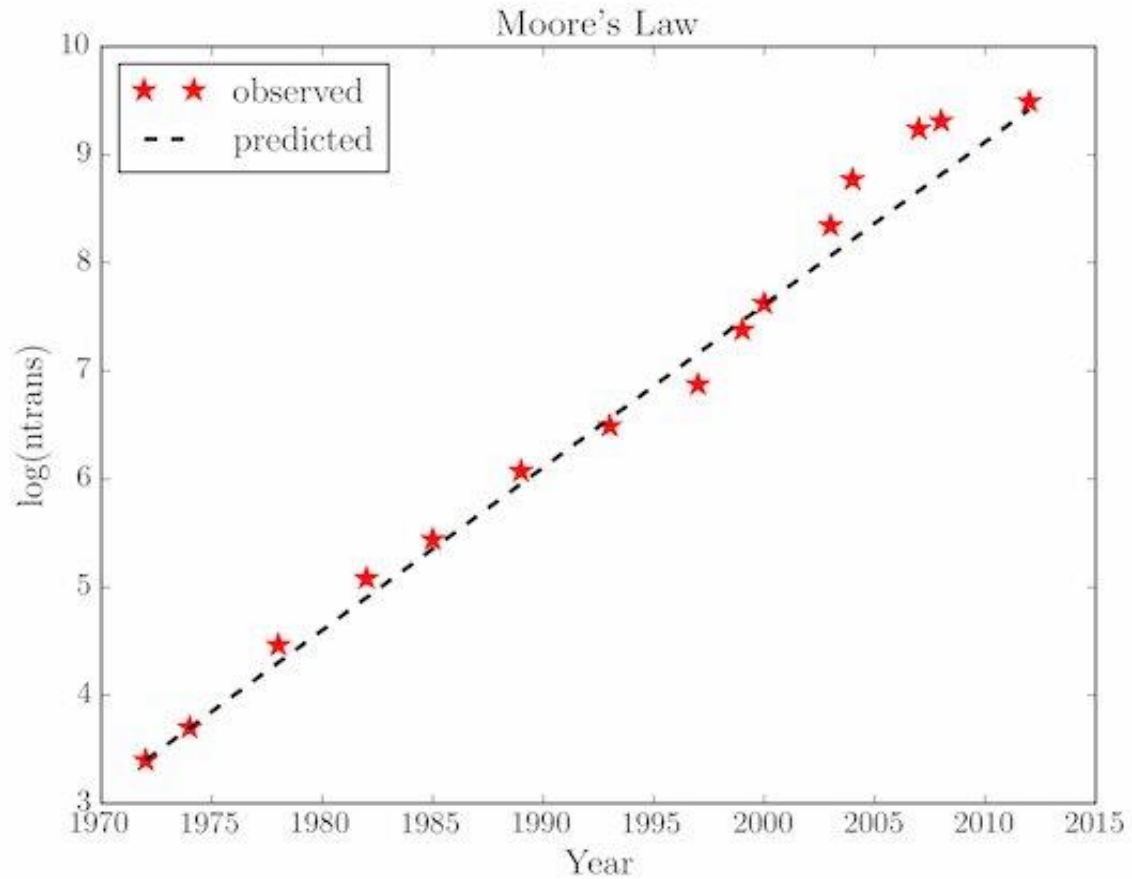
All nine pregnancies overlap: 9 babies after 9 months

Nine women cannot share one pregnancy to finish it in one month.

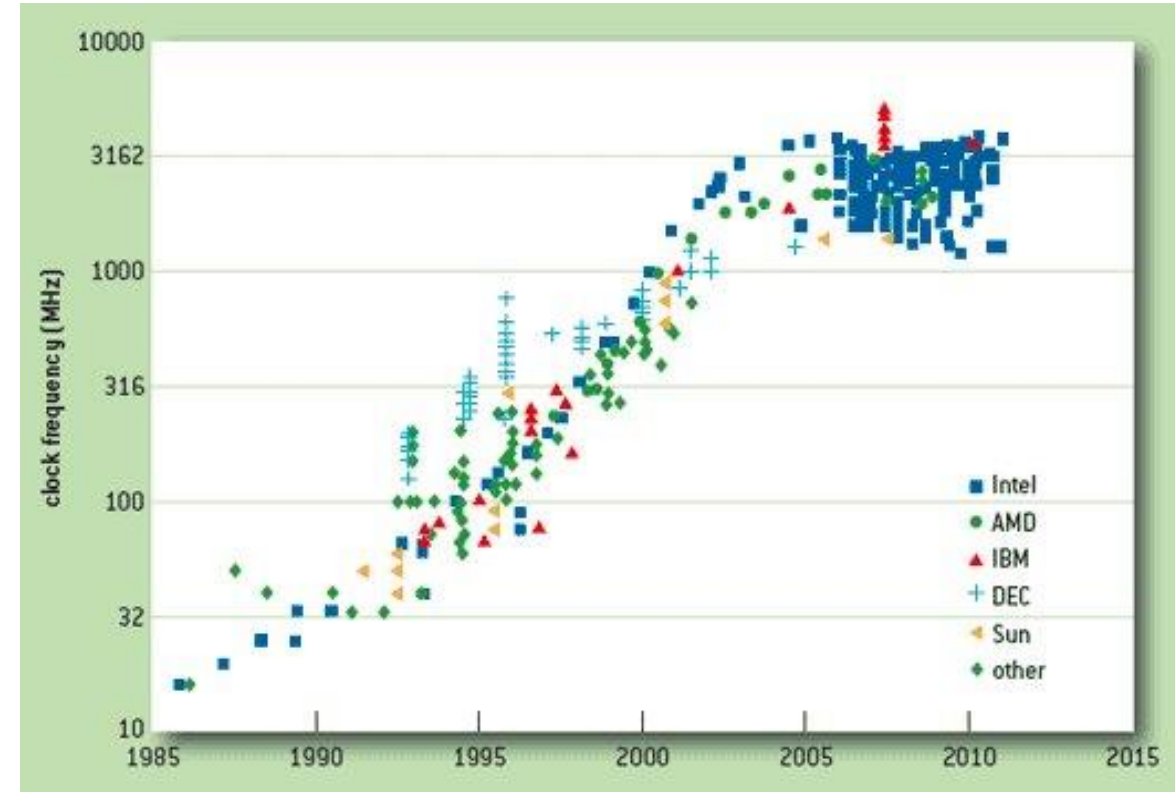
More concurrent tasks increase throughput. Each task still takes time.

Moore's law and Dennard scaling

Number of transistors



Clock Frequency



Moore's law and Dennard scaling

Moore's law: transistor count

- Historical expectation: roughly twice as many transistors per chip every two years.
- Further scaling faces physical and economic constraints.

Dennard scaling: power density

- Smaller transistors once allowed higher clock rates at similar power density.
- Around the mid-2000s, voltage scaling weakened. Power and heat constrained further frequency increases.

- CPUs: More transistors **do not** automatically accelerate sequential (Python) code.
- GPUs: Use **parallel work to increase throughput**, when the algorithm and data movement suit the hardware.

GPUs as accelerators

The hardware and programming tools vary by vendor.

NVIDIA



AMD



Intel



CUDA

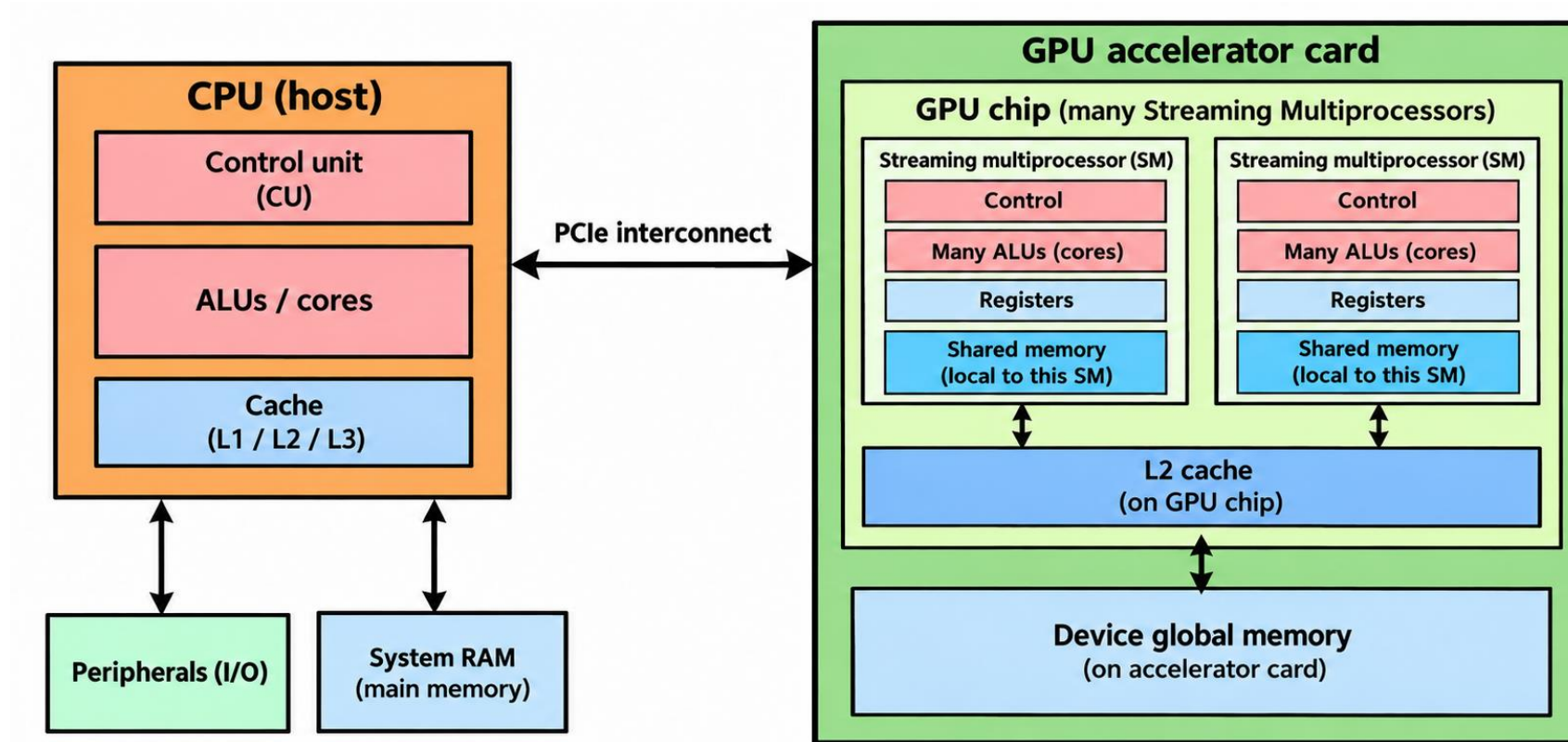
ROCm / HIP

oneAPI / SYCL

This course focuses on NVIDIA's CUDA model, using PyCUDA and Numba.

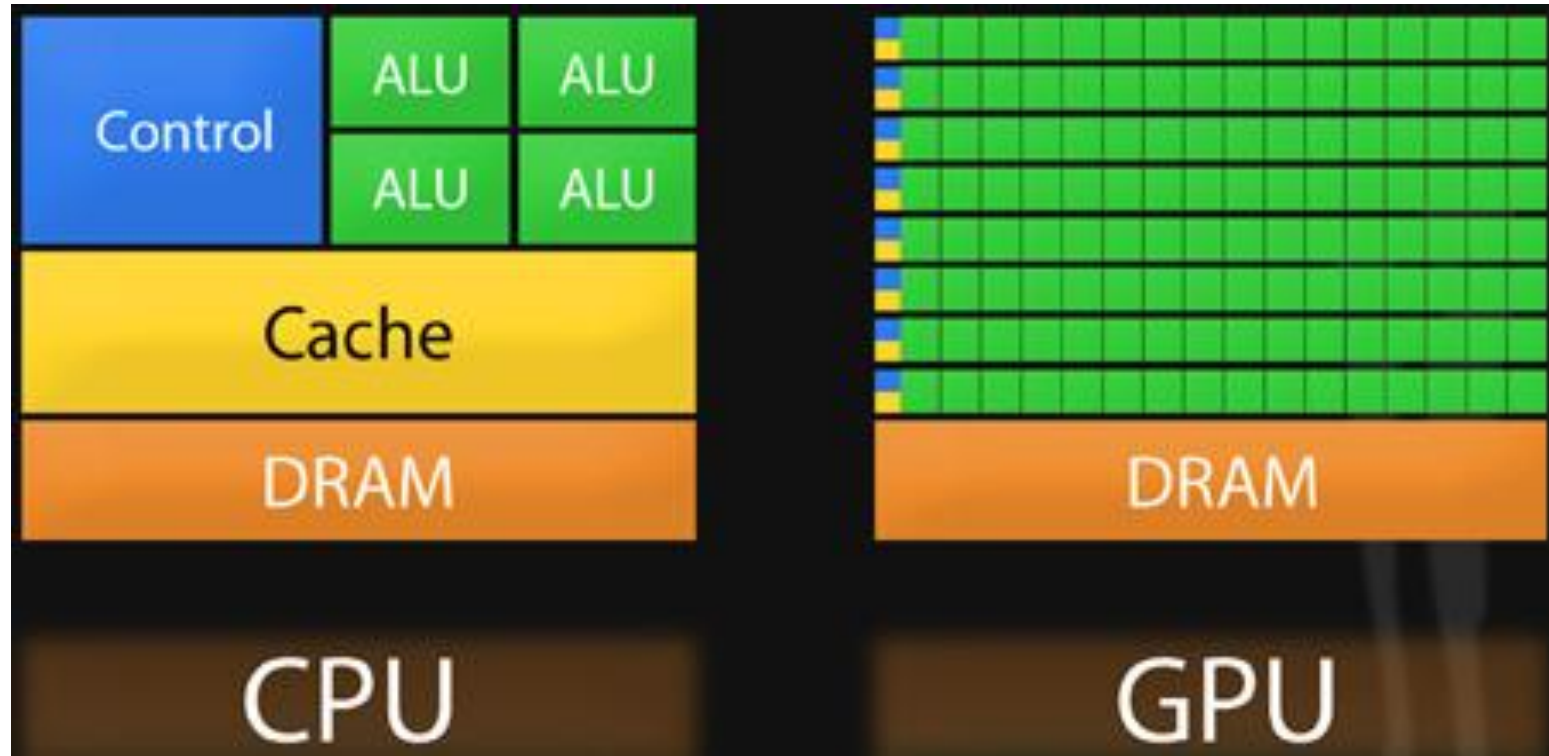
Parallelism and memory concepts apply across vendors. APIs and support differ.

CPU and GPU Architectures



The host and device exchange work and data

CPU and GPU resources - different goals



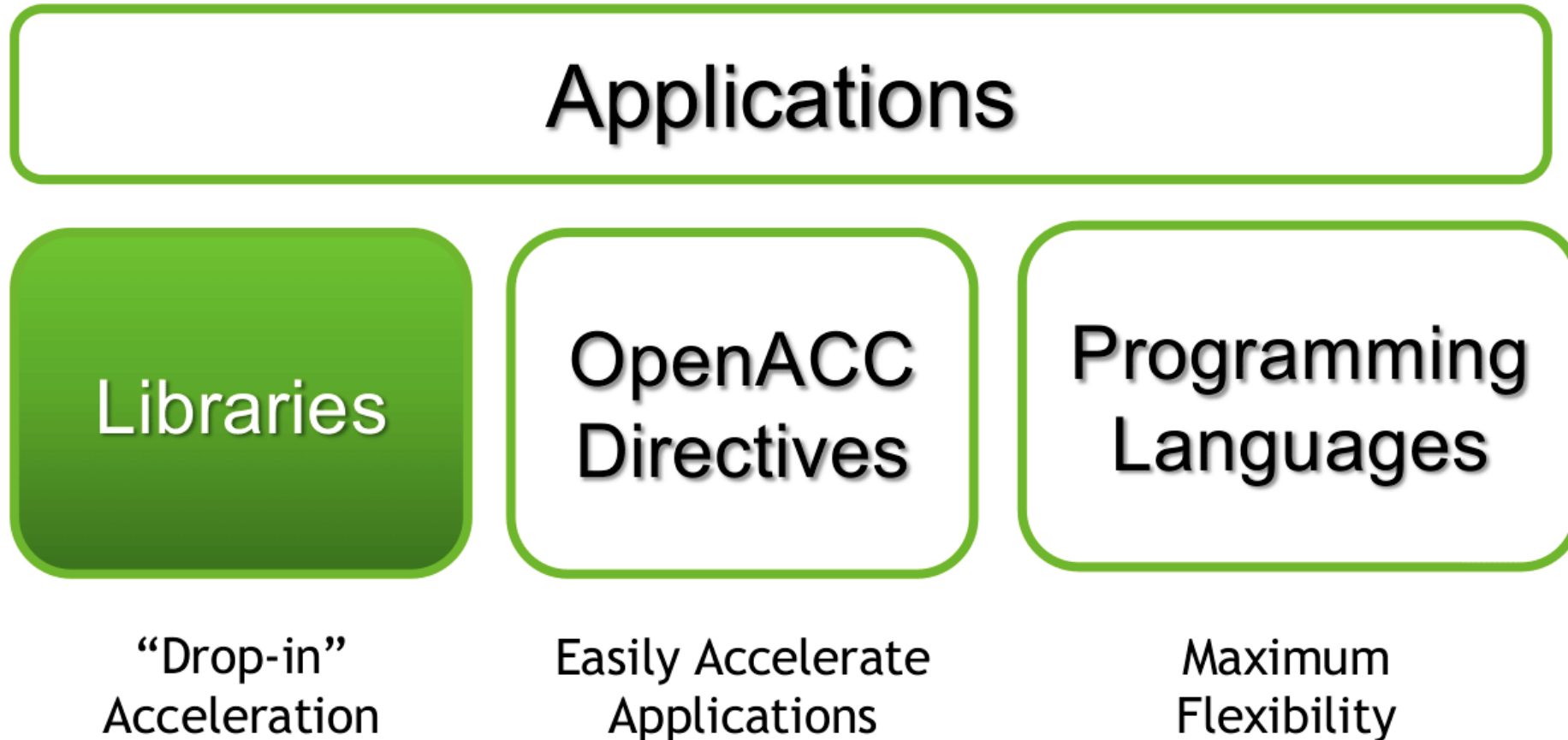
Latency and throughput answer different questions

Metric	Meaning	Example
Latency	Time for one task to finish	One dependent calculation
Throughput	Work completed per unit time	Many independent array elements

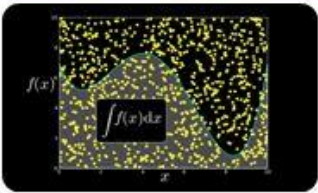
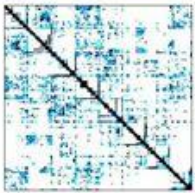
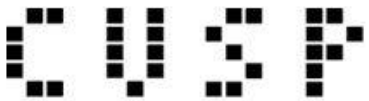
High GPU throughput needs sufficient parallel work. A small task may finish sooner on a CPU.

Libraries, directives, and kernels offer different entry points

3 Ways to Accelerate Applications



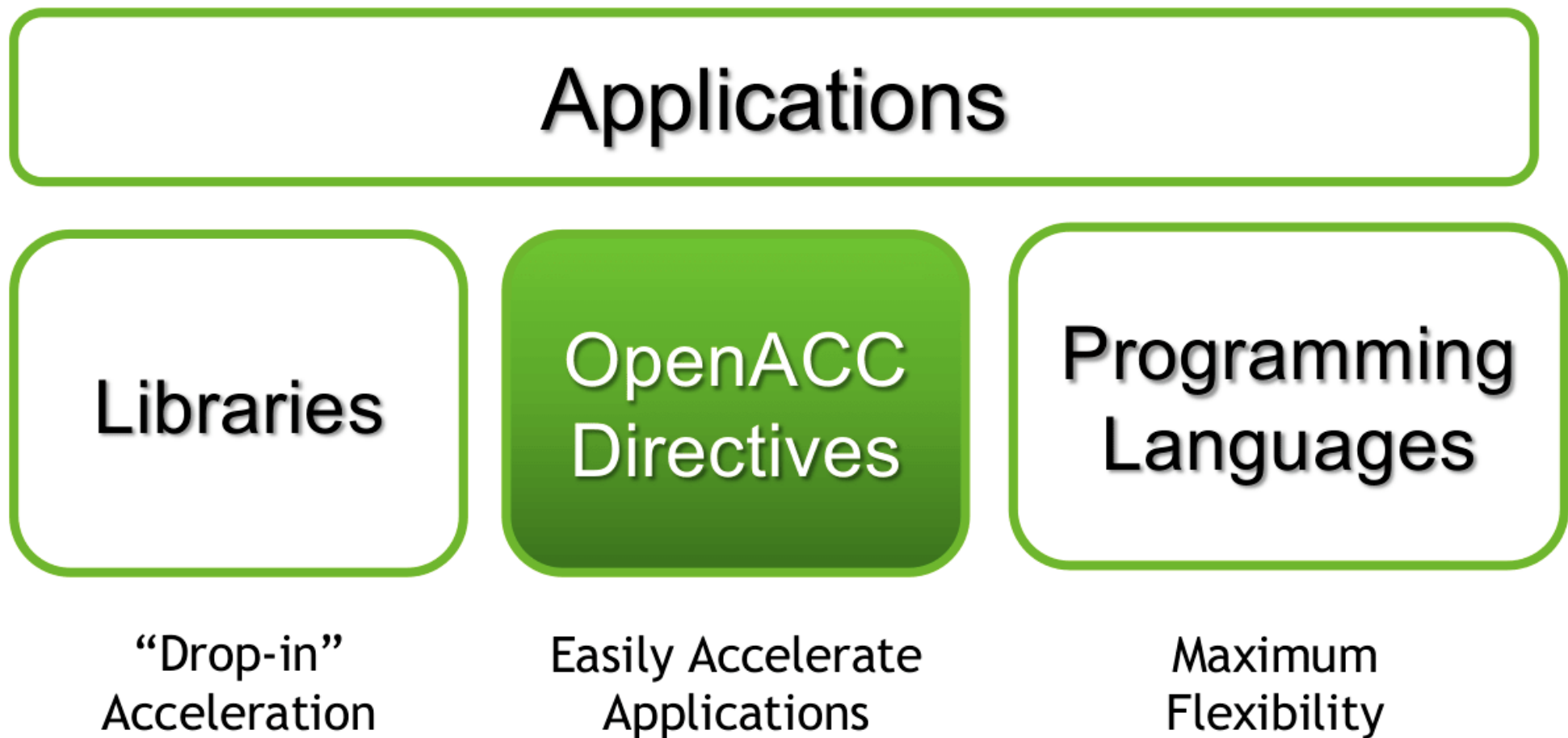
GPU Libraries

 NVIDIA cuBLAS	 NVIDIA cuRAND	 NVIDIA cuSPARSE	 NVIDIA NPP
 Vector Signal Image Processing	 GPU Accelerated Linear Algebra	 Matrix Algebra on GPU and Multicore 	 NVIDIA cuFFT
 IMSL Library	 Building-block Algorithms for CUDA	 Sparse Linear Algebra 	 C++ STL Features for CUDA 

GPU Accelerated Libraries
“Drop-in” Acceleration for Your Applications

GPU Usability

3 Ways to Accelerate Applications



OpenACC directives tell the compiler what to offload

Example: double every array element

```
// C code: a contains n numbers
#pragma acc parallel loop copy(a[0:n])
for (int i = 0; i < n; ++i) {
    a[i] = 2.0f * a[i];
}
```

parallel loop: iterations can run concurrently.

copy: move a to the device, then return its updated values.



Requires an OpenACC-capable compiler and independent loop iterations.

The compiler generates device code. PyCUDA and Numba expose kernels more directly.

OpenACC/OpenMP

```
#include <stdio.h>
#include <omp.h>

int main() {
    const int N = 1000;
    float a[N], b[N], c[N];

    // Initialize
    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = 2*i;
    }

    // Offload computation to GPU
    #pragma omp target teams distribute parallel for map(to:a[0:N], b[0:N]) map(from:c[0:N])
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }

    // Print a few results
    for (int i = 0; i < 5; i++) {
        printf("%f ", c[i]);
    }
    printf("\n");

    return 0;
}
```

```
#include <stdio.h>

int main() {
    const int N = 1000;
    float a[N], b[N], c[N];

    // Initialize
    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = 2*i;
    }

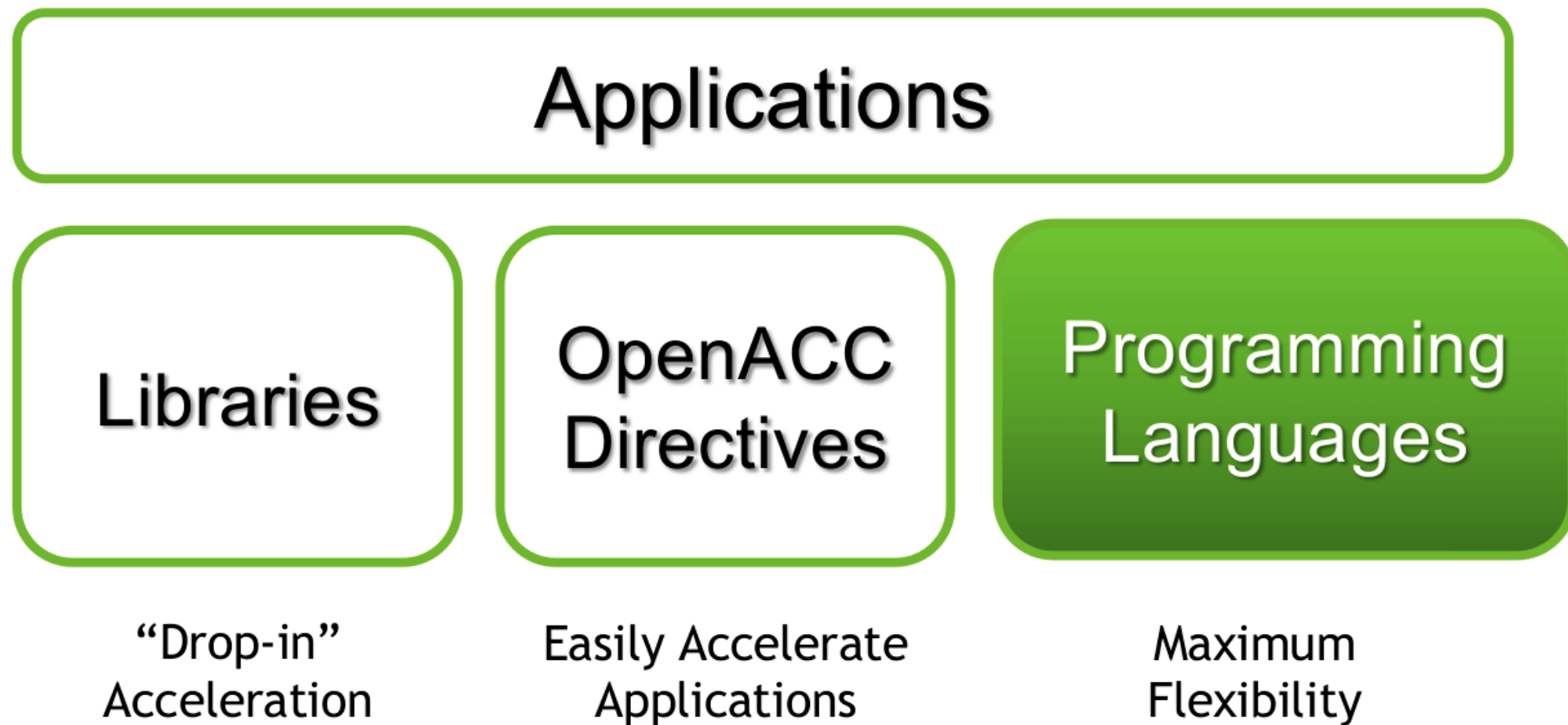
    // Offload computation to the GPU using OpenACC
    #pragma acc parallel loop copyin(a[0:N], b[0:N]) copyout(c[0:N])
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }

    // Print a few results
    for (int i = 0; i < 5; i++) {
        printf("%f ", c[i]);
    }
    printf("\n");

    return 0;
}
```

GPU Usability

3 Ways to Accelerate Applications



GPU Programming Languages

Numerical analytics ▶

MATLAB, Mathematica, LabVIEW

Fortran ▶

CUDA Fortran, OpenACC,
OpenMP4.5

C ▶

CUDA C, OpenCL, OpenACC,
OpenMP4.5

C++ ▶

CUDA C++, Thrust, OpenCL,
OpenACC/OpenMP4.5

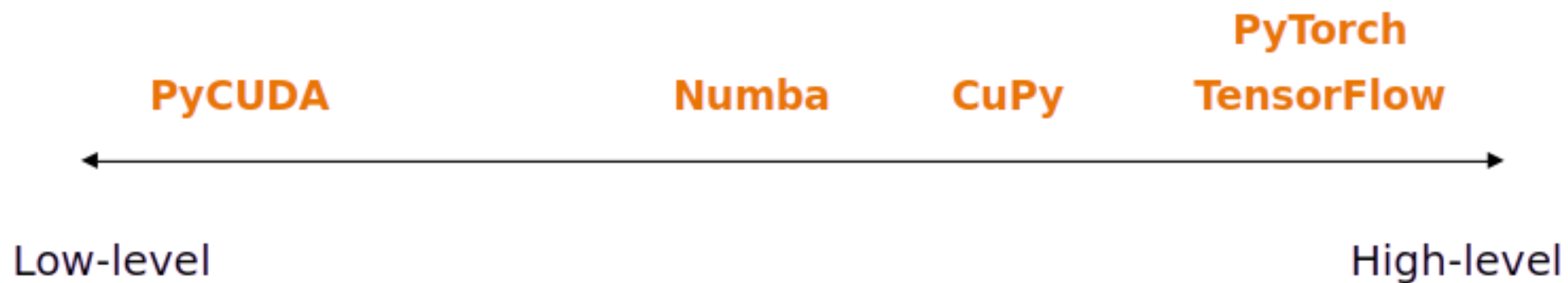
Python ▶

PyCUDA/PyOpenCL, Numba, ...

Core GPU Programming

- **Nvidia GPUs:**
 - CUDA, OpenCL, HIP
- **AMD GPUs:**
 - OpenCL, HIP
- **Intel GPUs:**
 - OpenCL, SYCL - oneAPI DPC++

Accessing to GPUs in Python



Python tools expose different levels of control

Tool	What you write	Typical use
CuPy	NumPy-like array expressions	Array operations and library routines
PyTorch / TensorFlow	Tensor operations and models	Machine learning workflows
Numba-CUDA	Restricted Python kernels	Custom numerical algorithms
PyCUDA	CUDA C/C++ kernels plus Python host code	Explicit CUDA control

PyCuda and Numba



PyCUDA

- It gives you easy, pythonic access to **NVIDIA's CUDA parallel computation API**
- There is more **flexibility** to write custom CUDA kernels
- For more information:
<https://documen.tician.de/pycuda/>

Numba

- It is an open-source **Just-In Time (JIT) compiler** that **translates** a subset of Python and Numpy into GPU machine code
- It uses a collection of decorators that can be applied to your functions to instruct Numba to compile them
- For more information:
<https://numba.pydata.org/>

PyCUDA

- PyCUDA makes CUDA's execution model explicit, providing a useful foundation for understanding Numba-CUDA.

Numba

- A thorough understanding of Numba-CUDA requires familiarity with the underlying CUDA execution model.

- Although Numba-CUDA simplifies kernel development in Python, effective GPU programming still requires an understanding of threads, blocks, grids, and memory management. Performance improvement also requires more in-depth approaches
 - That is why we will start this course by addressing PyCuda first

The environment



A working GPU allocation is the first prerequisite

Confirm a GPU session, the visible device, available memory, and driver/toolkit compatibility.

Use the course-approved environment and notebook kernel to verify that the NVIDIA GPU is available, that the driver is working, and that the GPU has sufficient resources to run your application.

Run:

```
$ nvidia-smi
```

Snellius Supercomputer

- 72 GPU nodes with 4 A100 NVIDIA GPUs; 288 A100 GPUs
- 88 GPU nodes with 4 H100 NVIDIA GPUs; 352 H100 GPUs
- **Example #1**
 - 1 - Try to implement parallel vector addition using PyCUDA
- In total 640 GPUs

GPU Node

- 4 NVIDIA A100 GPUs per node
 - Multiprocessors: 108
 - Streaming cores: 6912
 - Tensor Cores: 432
 - Global memory: 40 GB
- MIG partitions: 1/7th of A100 GPUs
- One GPU is shared among 7 people
- Note that you have around 5 GB memory:
 - Matrix $(35,000 * 35,000) = 35,000 * 35,000 * 4 \approx 5 \text{ GB}$

Let's start programming



Vector Addition

- **Example #1**

- 1 - Try to implement parallel vector addition using PyCUDA
- Compare its execution time to the CPU version (sequential)

Calculate Global Index (1D grid, 1D block)

Global Thread ID

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

threadIdx.x

threadIdx.x

threadIdx.x

threadIdx.x

0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

blockIdx.x = 0

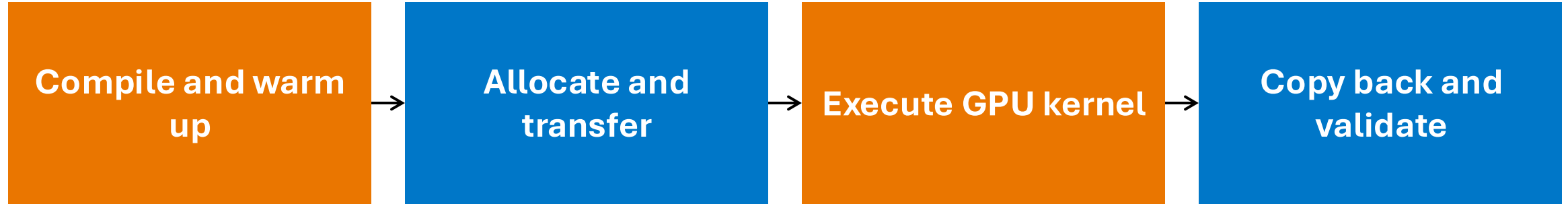
blockIdx.x = 1

blockIdx.x = 2

blockIdx.x = 3

- Global Thread ID: $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$
- For global thread ID 26:
 - $\text{blockIdx.x} = 3$
 - $\text{blockDim.x} = 8$
 - $\text{threadIdx.x} = 2$
 - Global thread ID = $3 * 8 + 2 = 26$

Kernel time and total time measure different costs

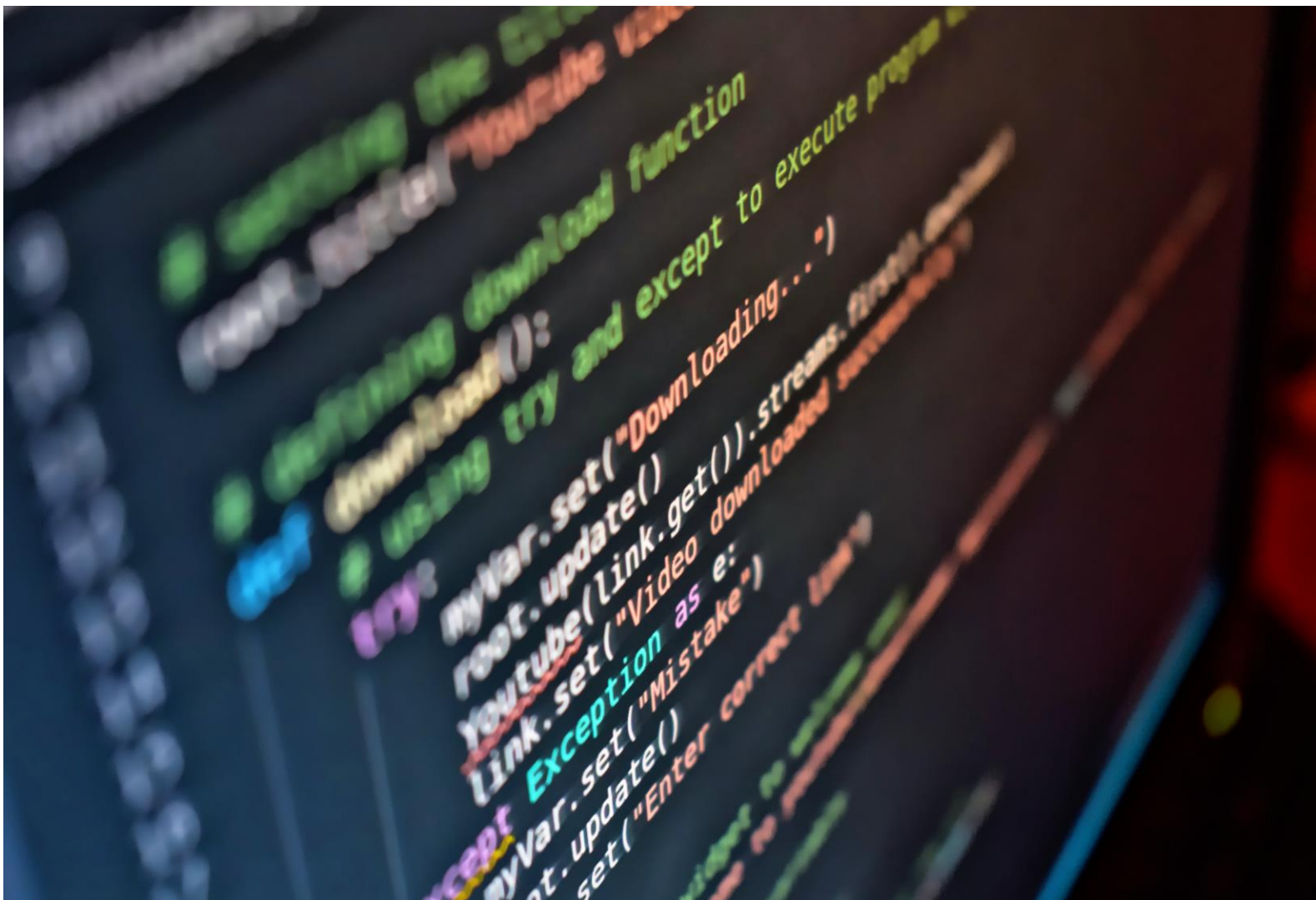


Kernel time uses device events around execution.

Total time uses a host clock around the chosen pipeline and waits for completion.
Compilation is a separate cold-start cost.

Results

How can we improve pycuda implementation?



Automatic Data Transfer

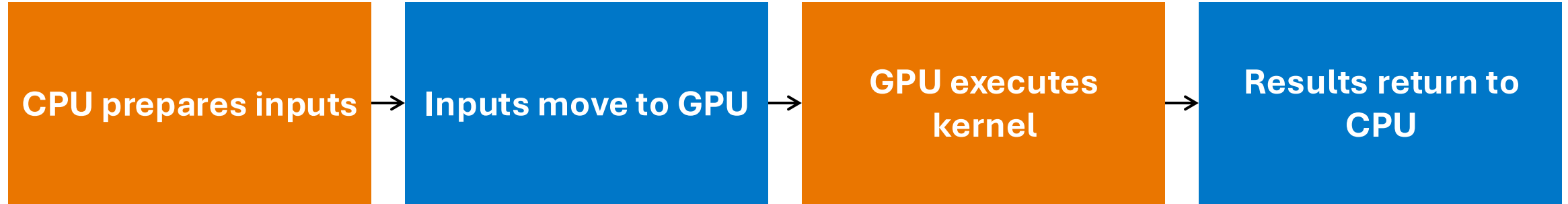
- Automatic data transfer using PyCUDA driver:
 - In()
 - Out()
 - InOut()
- PyCUDA programs become simpler

Results

Discussion



GPU acceleration changes the execution pipeline

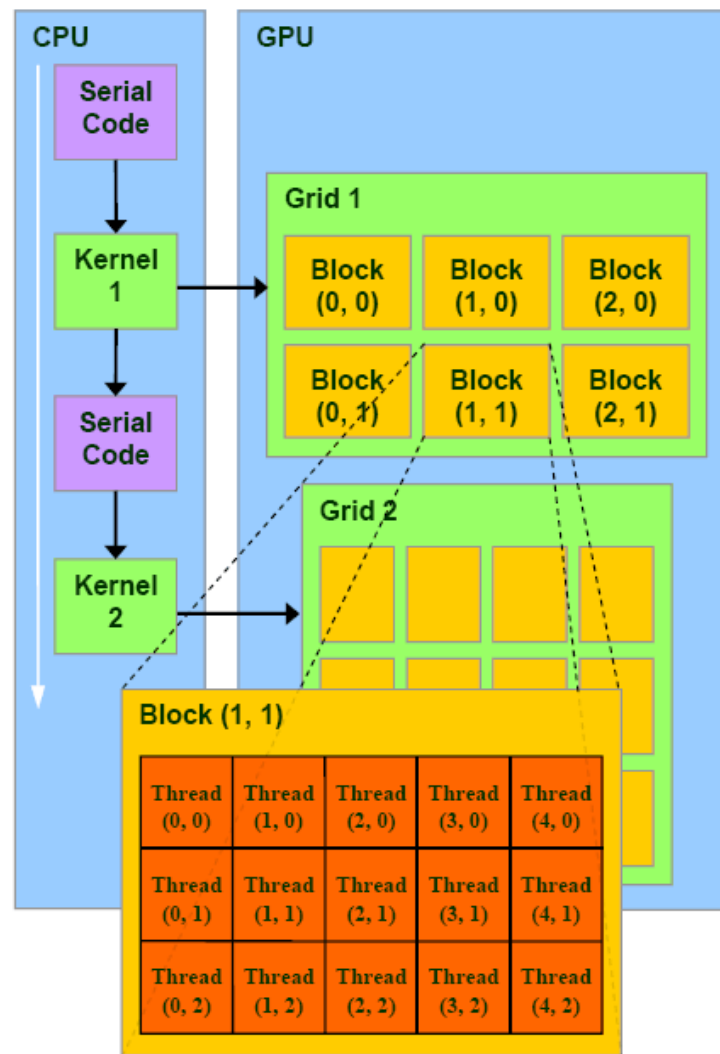


- Reuse device arrays across several operations to avoid repeated transfers.
- Measure both the complete pipeline and the kernel alone.

CUDA separates host code from device kernels

Term	Role
Host	CPU code allocates data and launches kernels
Device	GPU executes many instances of a kernel
Kernel	Function whose threads write output arrays
Launch configuration	Number of blocks and threads per block

CUDA Programming Model



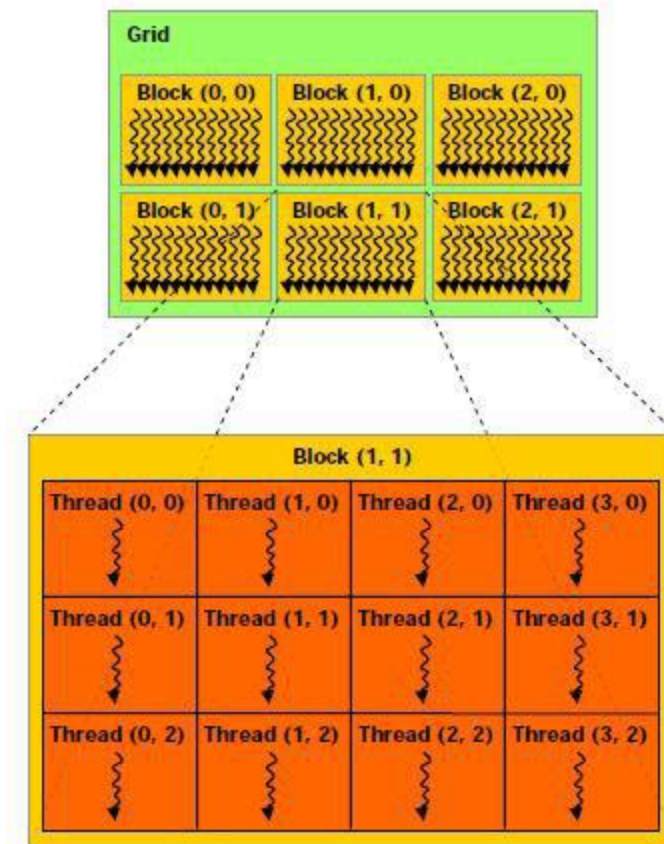
CUDA Programming Model

Threads are organized into two hierarchical levels:

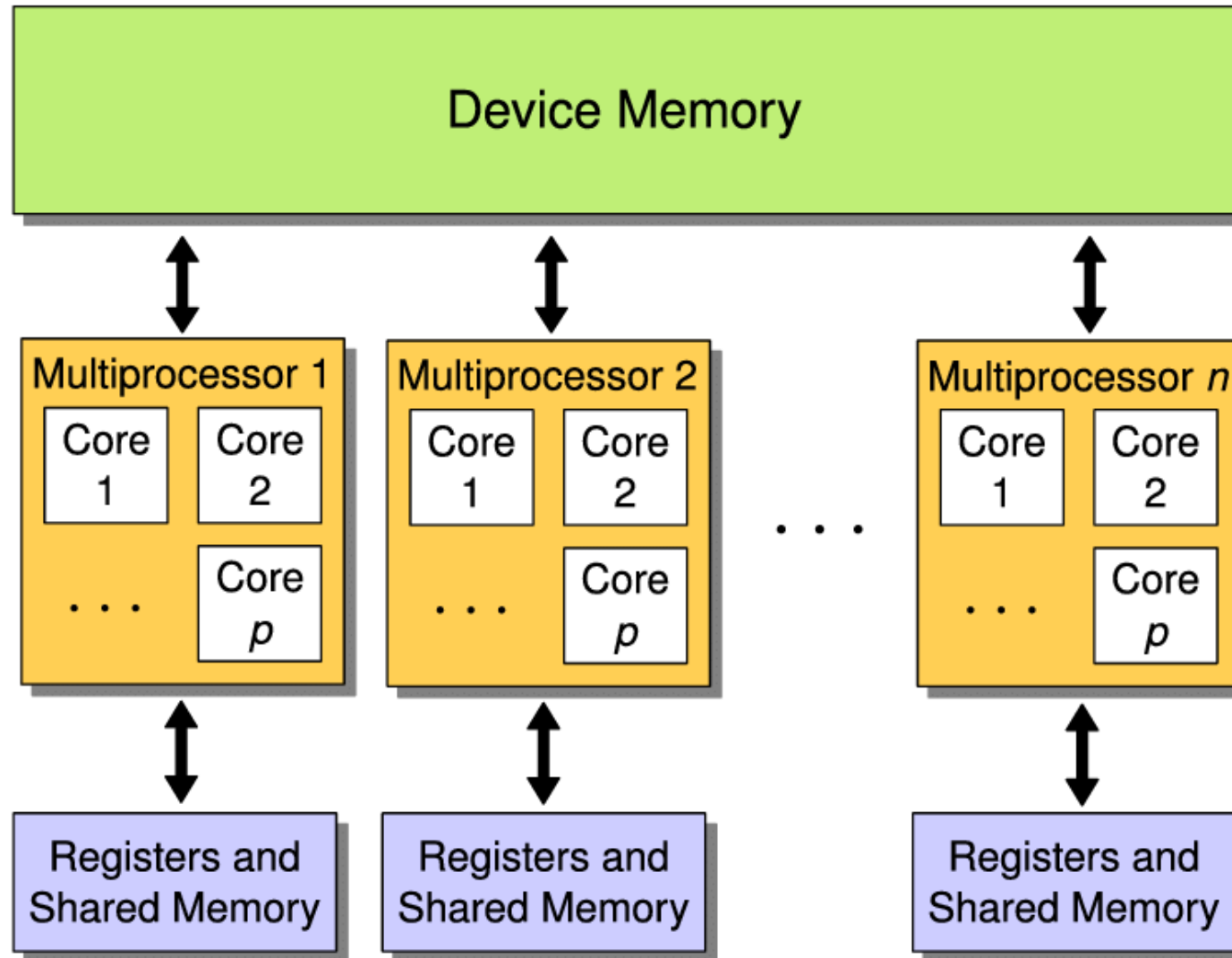
- Threads are grouped into blocks
- Blocks are grouped into grids

Blocks and grids can be

- 1D, 2D and 3D

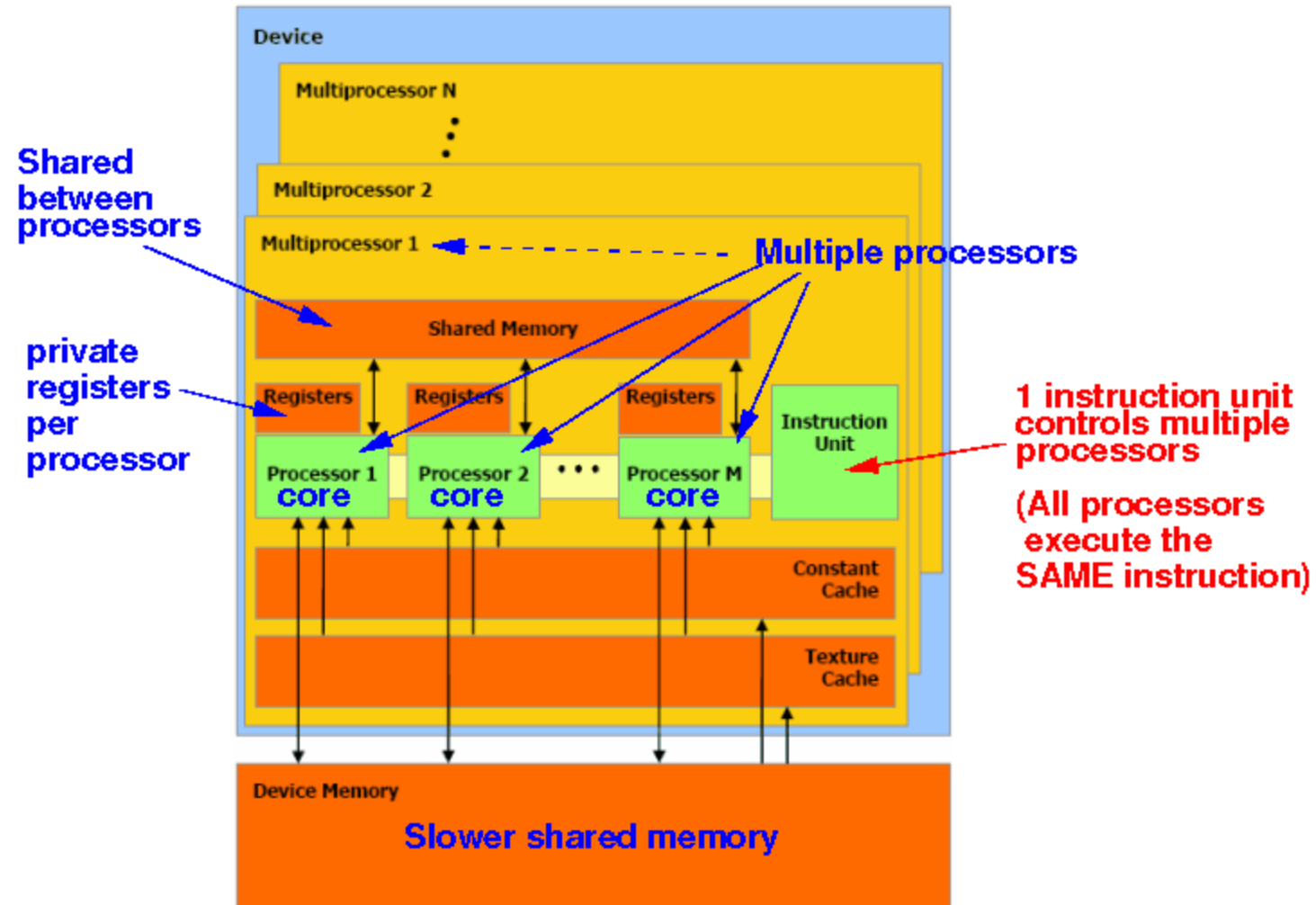


Streaming multiprocessors execute groups of threads



The hardware hierarchy differs from the launch hierarchy

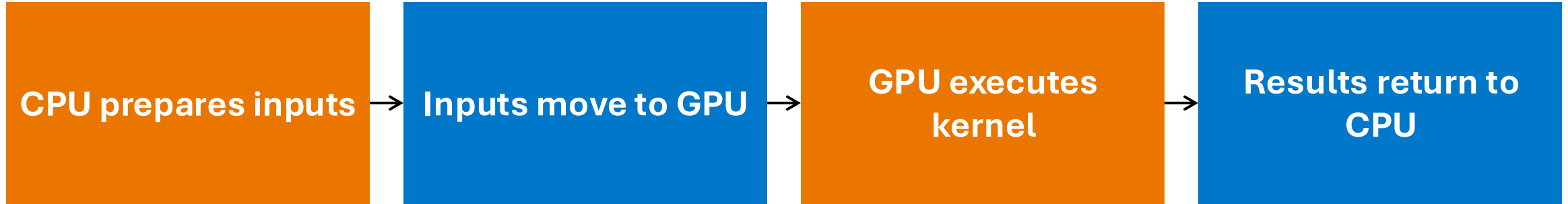
GPU device:



Memory scope determines who can exchange data

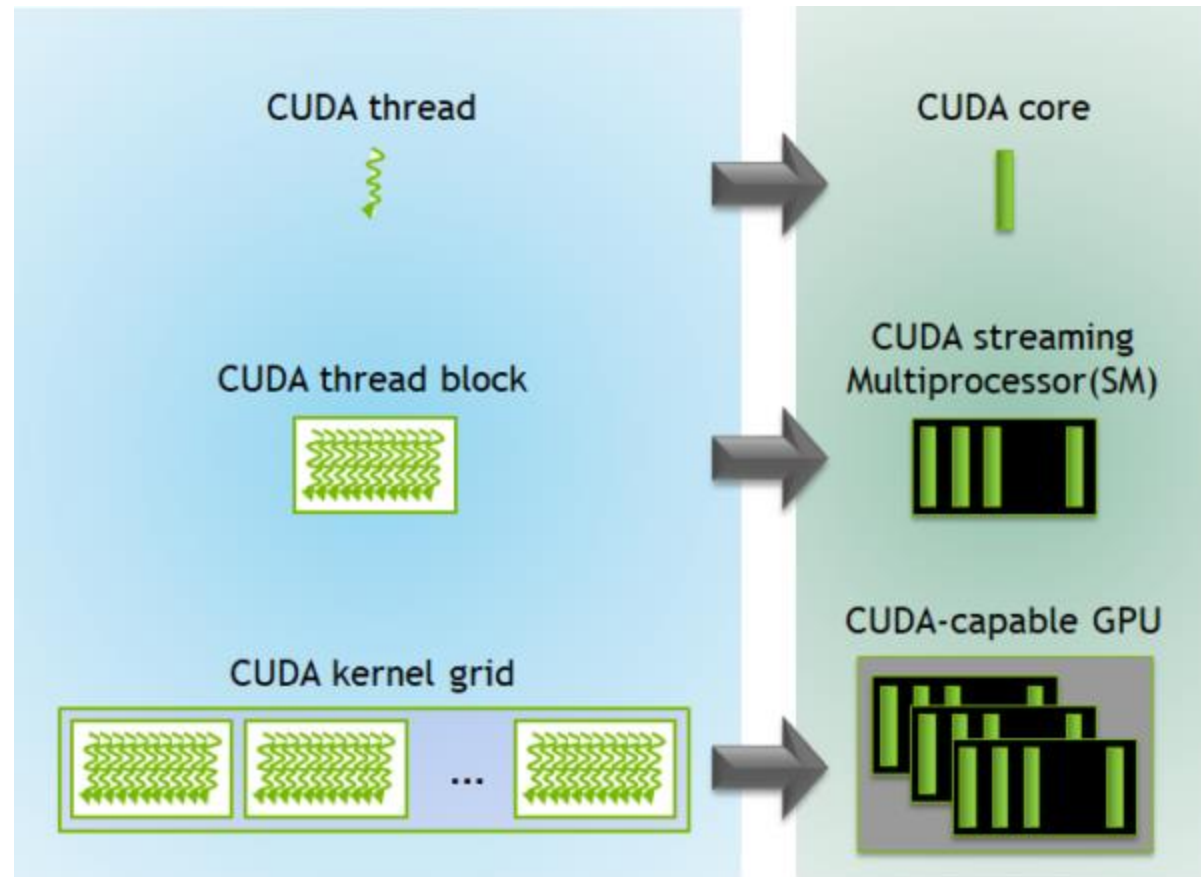
Storage	Accessible by	Lifetime / caution
Registers	One thread	Thread-private, limited resource
Local memory	One thread	Device-memory backed, may include spills
Shared memory	Threads in one block	Block lifetime, needs coordination
Global device memory	Threads across the grid	Persists across launches while allocated
Host RAM	CPU code	Requires transfer or another access mechanism

GPU acceleration changes the execution pipeline

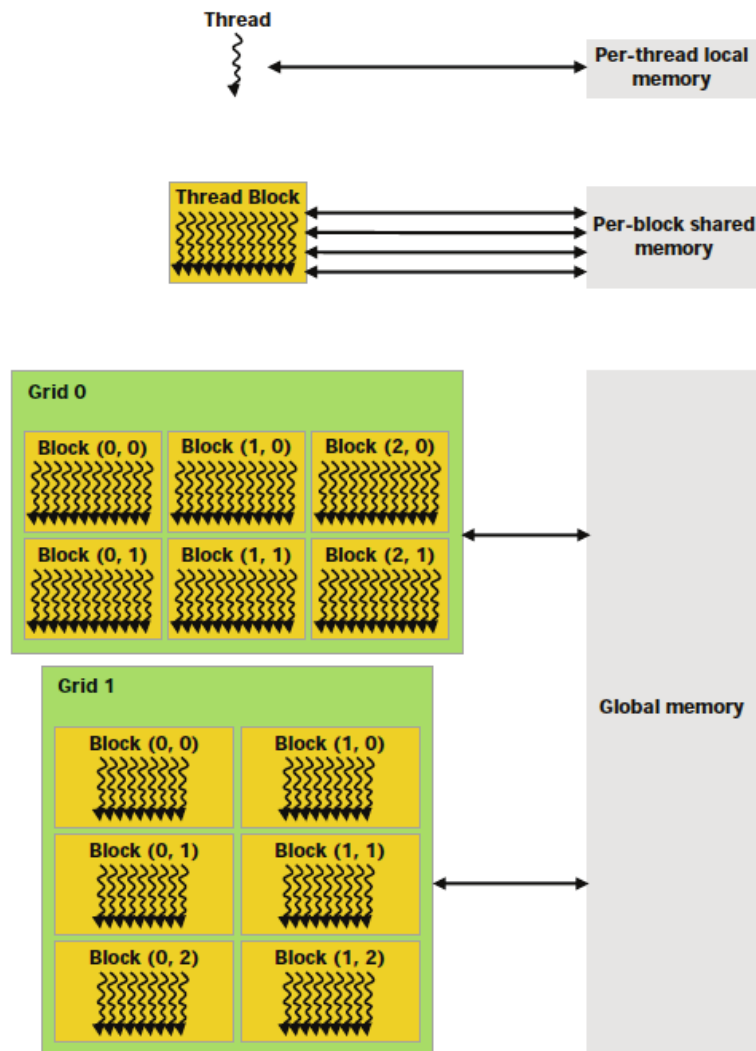


- Reuse device arrays across several operations to avoid repeated transfers.
- Measure both the complete pipeline and the kernel alone.

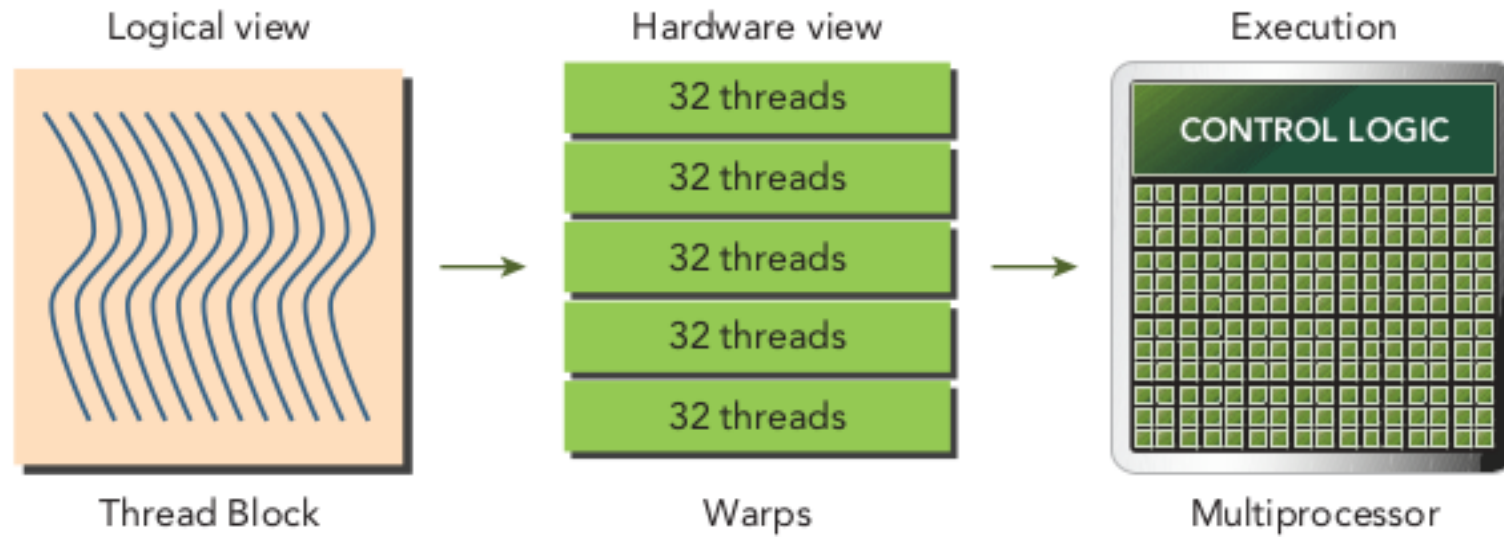
CUDA Execution Model



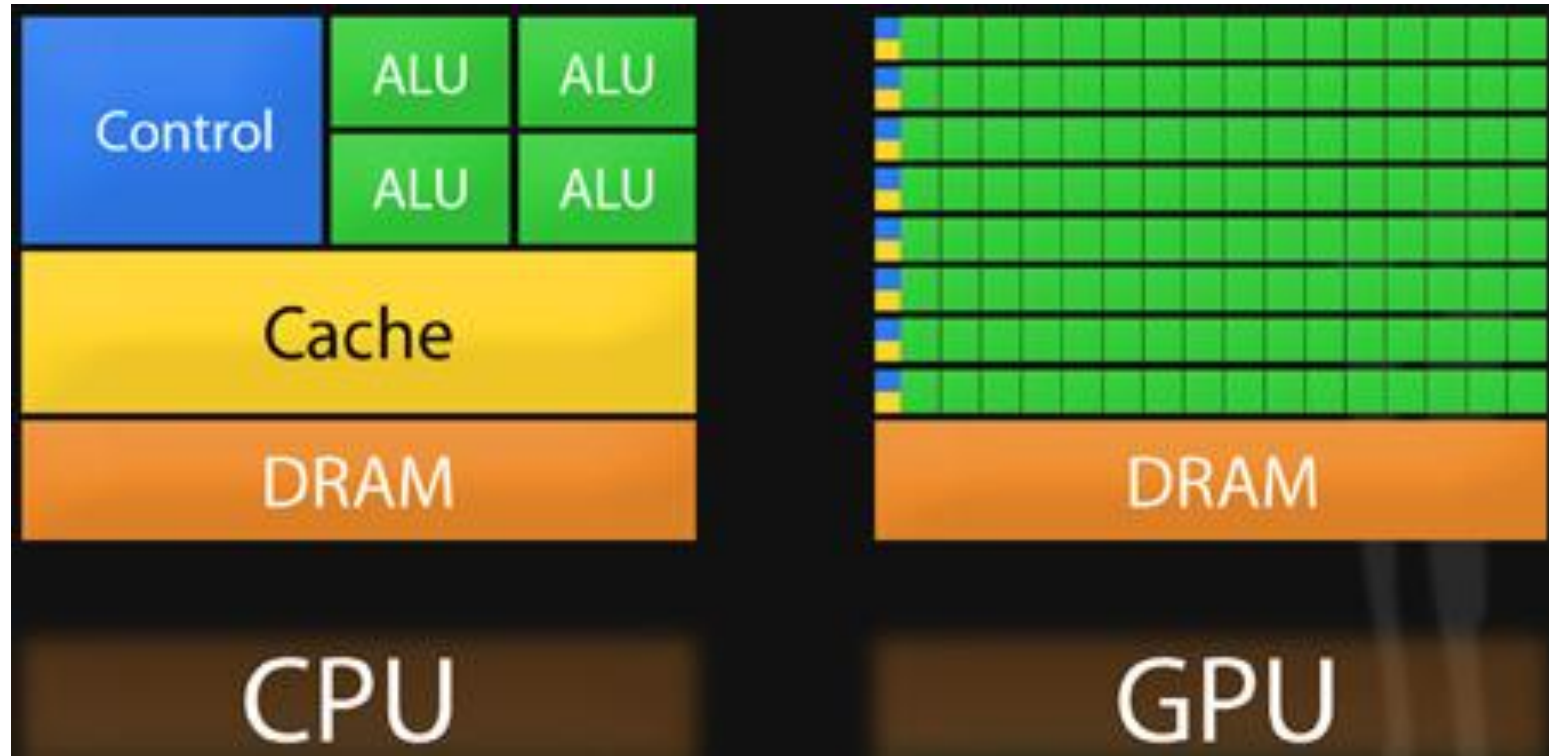
CUDA Execution Model



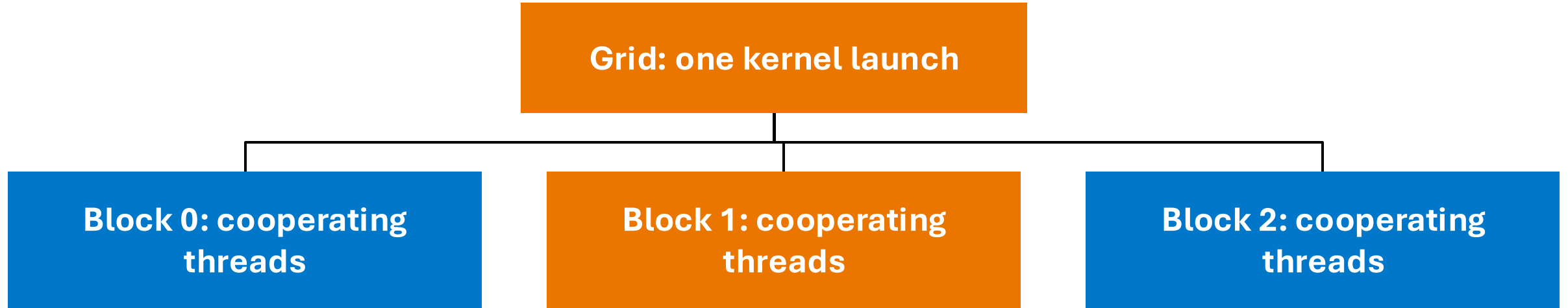
CUDA Execution Model



CPU and GPU resources - RECAP



A grid contains blocks, and each block contains threads

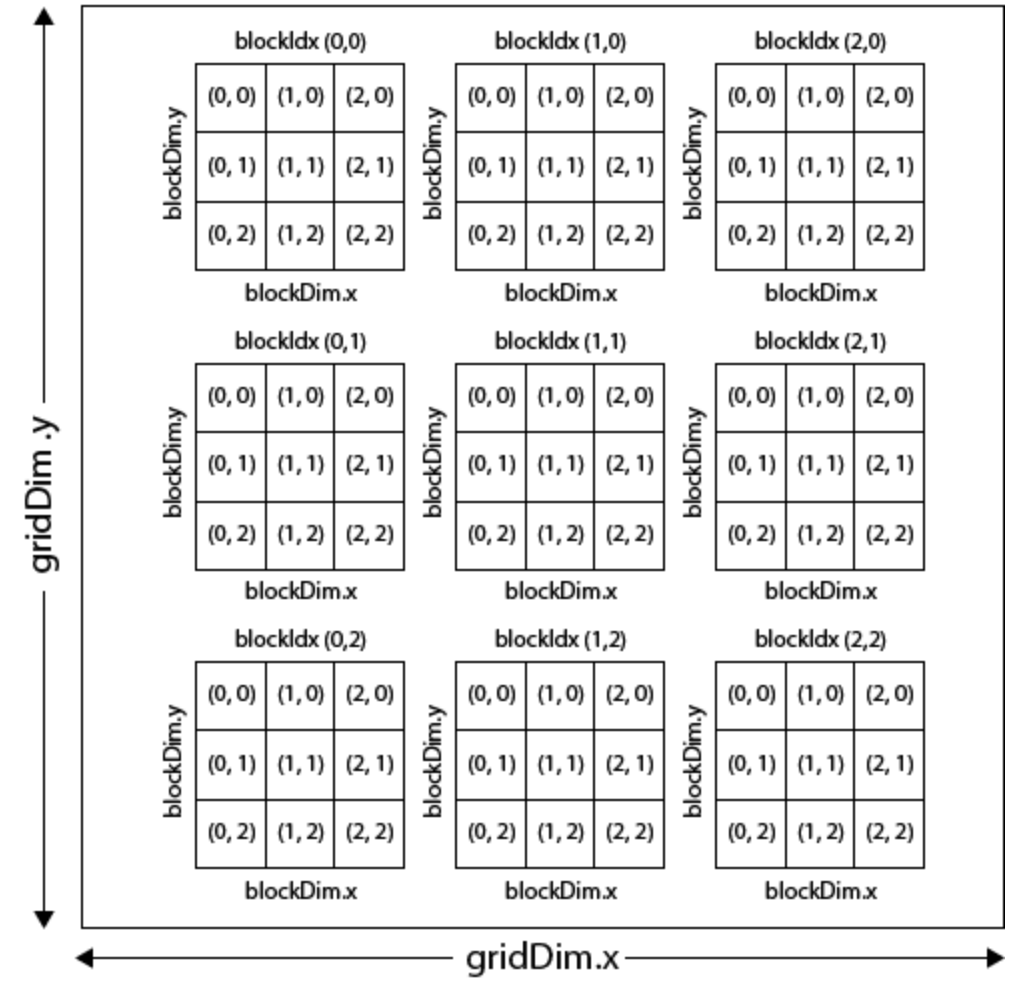


- Each thread has a local index inside its block.
- Blocks have indices inside the grid. Blocks can execute in any order.

A two-dimensional launch uses x and y coordinates

- `gridDim.x = 3`
- `gridDim.y = 3`
- `blockDim.x = 3`
- `blockDim.y = 3`

CUDA Grid



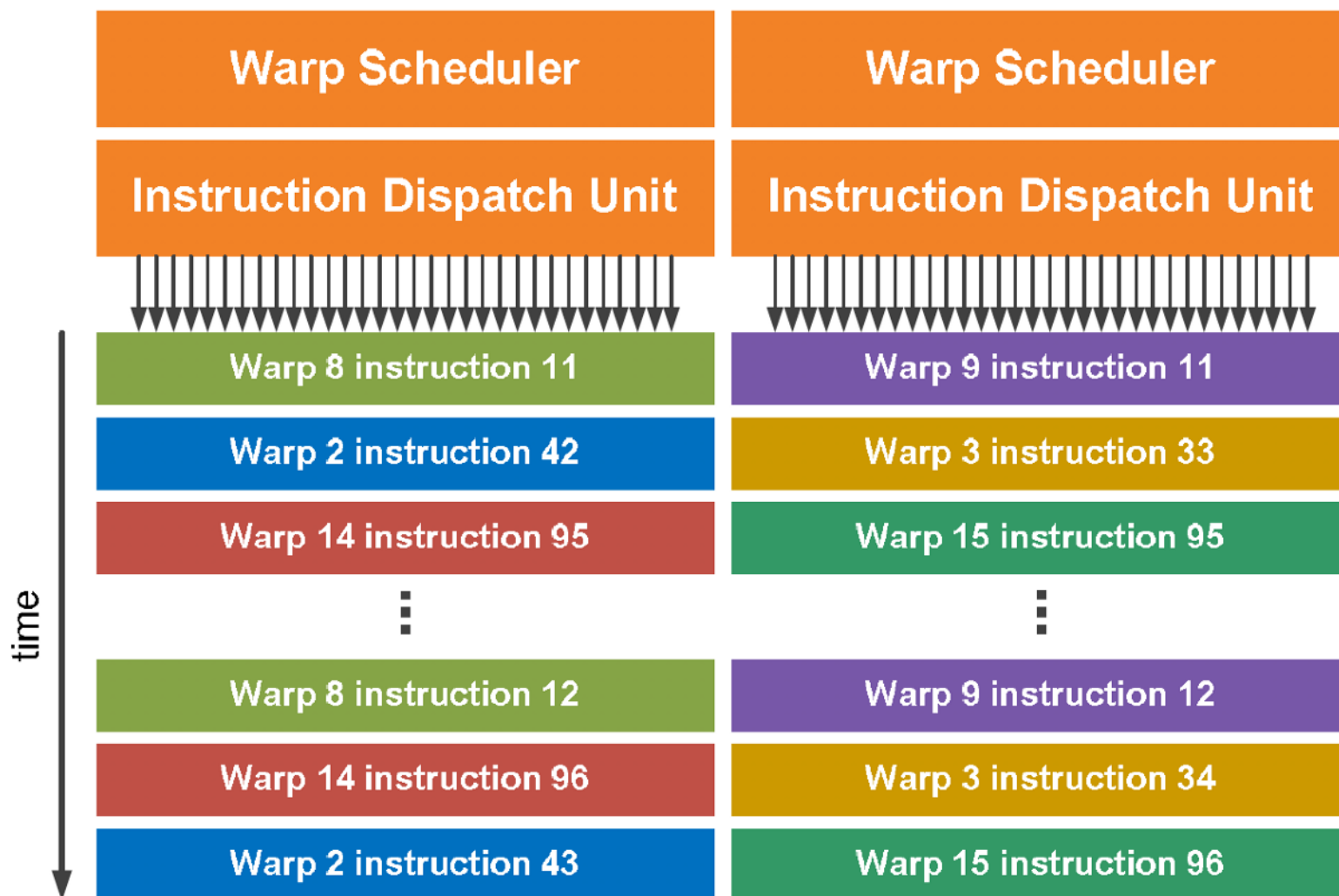
A global index combines the block and thread indices

Global Thread ID

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
threadIdx.x								threadIdx.x								threadIdx.x								threadIdx.x							
0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
blockIdx.x = 0								blockIdx.x = 1								blockIdx.x = 2								blockIdx.x = 3							

- Global Thread ID: $\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$
- For global thread ID 26:
 - $\text{blockIdx.x} = 3$
 - $\text{blockDim.x} = 8$
 - $\text{threadIdx.x} = 2$
 - Global thread ID = $3 * 8 + 2 = 26$

CUDA Execution Model



Synchronization in CUDA

- **There is** a mechanism to synchronize all threads in a block:
 - Built-in function `__syncthreads()`
- **There is no** mechanism to synchronize all threads across all blocks
 - Decouple the kernel into two separate kernels

Let's start programming



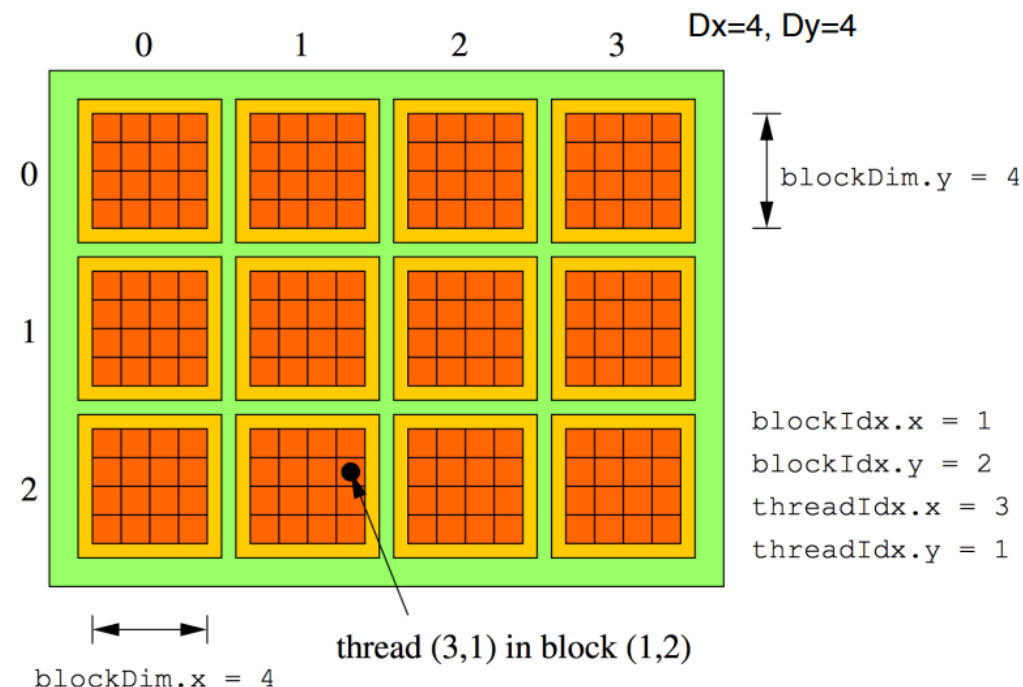
Matrix Addition

- **Example #2**

- Try to implement parallel matrix addition using **Pycuda**
- Compare its execution time to the CPU version (sequential)

Calculate Global Index (2D grid, 2D block)

- Matrix 12*16



- Global Thread ID:

- $row = blockDim.y * blockIdx.y + threadIdx.y = 2 * 4 + 1 = 9$
- $column = blockDim.x * blockIdx.x + threadIdx.x = 1 * 4 + 3 = 7$

Row-Major Flattening of a Matrix

- Matrix 3*3
- For each element (row, col):
 - New ID = row * (No of col) + col
- For instance element “5” in location (1, 2):
 - New ID = 1 * 3 + 2 = 5

How we see a 2D array

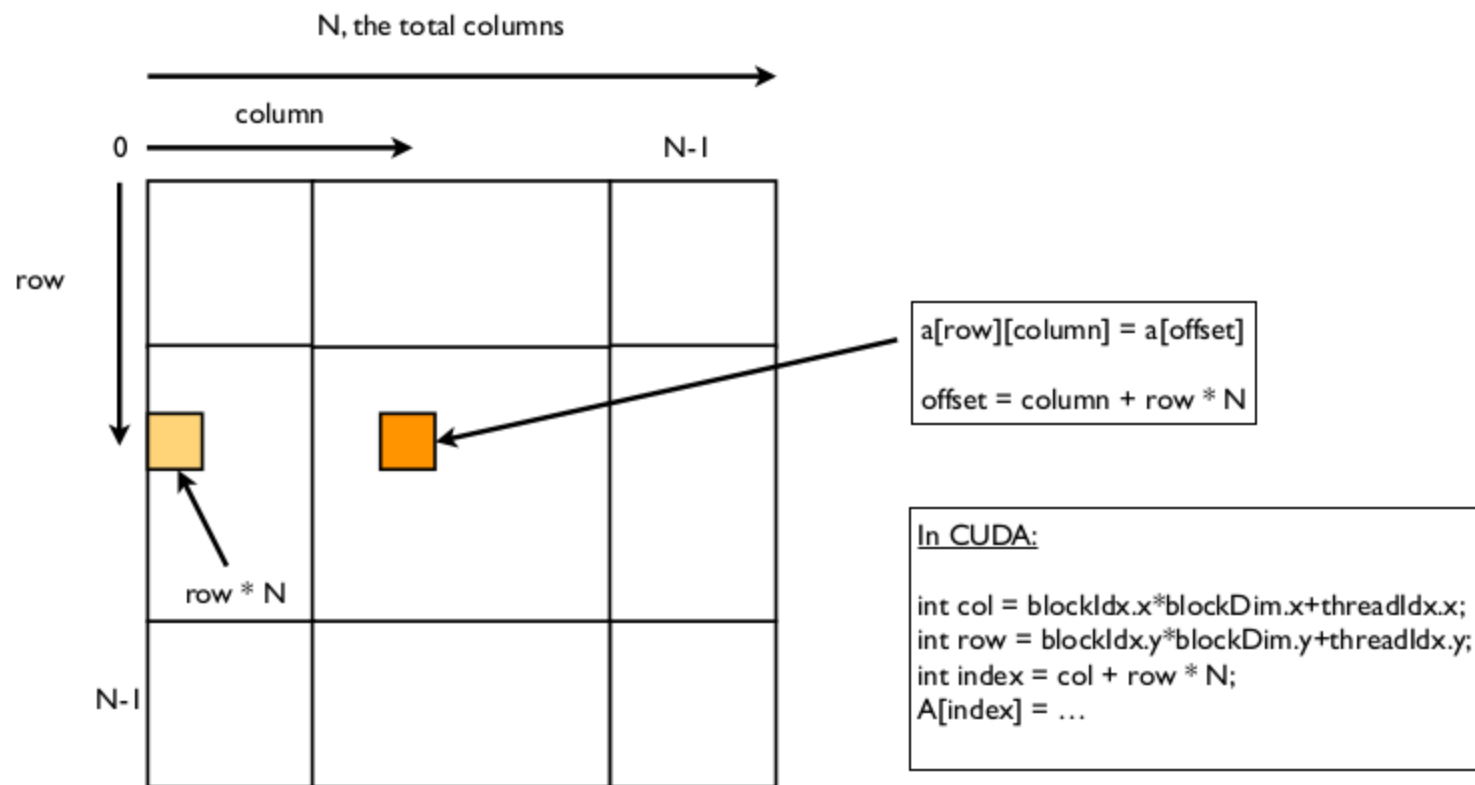
0	1	2
3	4	5
6	7	8

How it's stored in memory

↓

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

Row-Major Flattening of a Matrix



When Numba is Recommended?

- Scientific code: numerically orientated
- Code is based on numerical Python (numpy)
- Data structures are type-consistent, not mixed
- Code contains several loops that iterate over data
- Development time more important than tuning performance and efficiency

Let's start programming



Vector Addition

- **Example #3**

- Try to implement parallel vector addition using **Numba**
- Compare its performance and style to the Pycuda version

When Numba is Recommended?

- Scientific code: numerically orientated
- Code is based on numerical Python (numpy)
- Data structures are type-consistent, not mixed
- Code contains several loops that iterate over data
- Development time more important than tuning performance and efficiency

Checkpoint: which threads can cooperate directly?

1. Can two ordinary blocks safely rely on execution order?
2. Can one block read another block's shared array?
3. What happens if a thread exits before a required barrier?

Numba assigns one vector element to each thread

```
from numba import cuda

@cuda.jit
def add_numba(a, b, c):
    i = cuda.grid(1)
    if i < c.size:
        c[i] = a[i] + b[i]
```

@cuda.jit compiles the kernel for the GPU.

cuda.grid(1) computes the global thread index.

The kernel writes c. It does not return a host array.

PyCUDA expresses the same kernel in CUDA C/C++

```
import pycuda.autoinit
import pycuda.driver as drv
from pycuda.compiler import SourceModule

mod = SourceModule("""
__global__ void add(const float *a, const float *b,
                    float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
""")
add_pycuda = mod.get_function("add")
```

Python manages the context and compilation.

Each CUDA thread executes the kernel body once.

PyCUDA and Numba use the same CUDA concepts

Concept	PyCUDA kernel	Numba-CUDA kernel
Global 1D index	<code>blockIdx.x * blockDim.x + threadIdx.x</code>	<code>cuda.grid(1)</code>
Block size	<code>blockDim.x</code>	<code>cuda.blockDim.x</code>
Local index	<code>threadIdx.x</code>	<code>cuda.threadIdx.x</code>
Block barrier	<code>__syncthreads()</code>	<code>cuda.syncthreads()</code>
Shared storage	<code>__shared__ float s[T]</code>	<code>cuda.shared.array(T, float32)</code>

Let's start programming



Matrix Matrix Multiplication

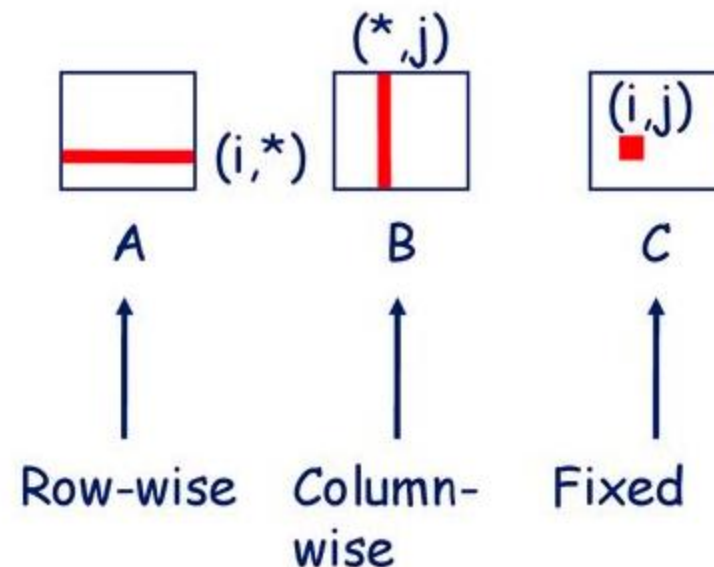
- **Example #4**

- 1 - Try to implement matrix multiplication in parallel using PyCUDA
- 2 - Try to implement matrix multiplication in parallel using Numba GPU
- 3 - Try to implement matrix multiplication in parallel using Numba CPU
- 4 - Compare execution times
 - Consider the @ operator

Sequential Matrix Multiplication

```
for i in range(N):  
    for j in range(N):  
        for k in range(N):  
            C[i][j] += A[i][k] * B[k][j]
```

Inner loop:



$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

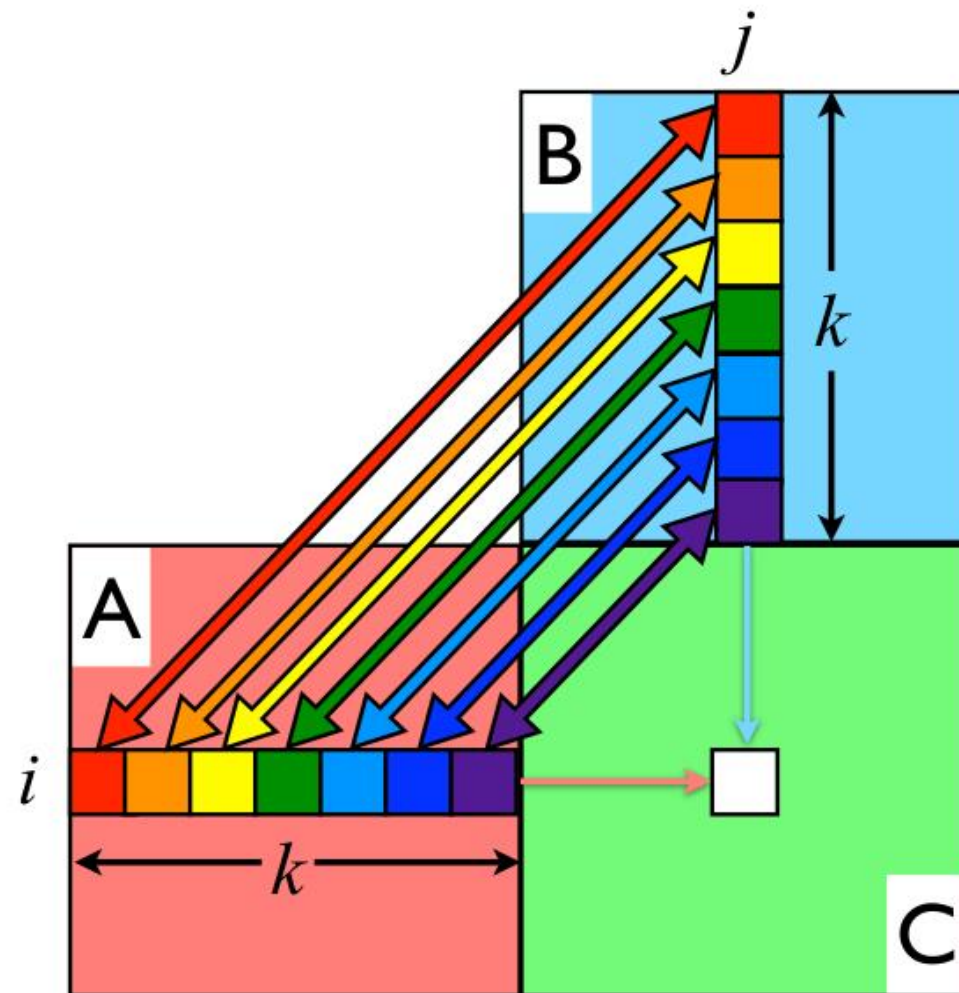
A B C

Parallel Matrix Multiplication

```
int k, sum = 0;

int col = threadIdx.x + blockDim.x * blockIdx.x;
int row = threadIdx.y + blockDim.y * blockIdx.y;
int id = row * N + col;

if (col < N && row < N){
    for(k = 0; k < N; k++){
        sum += A[row * N + k] * B[k * N + col];
    }
    C[id] = sum;
}
```



Let's start programming

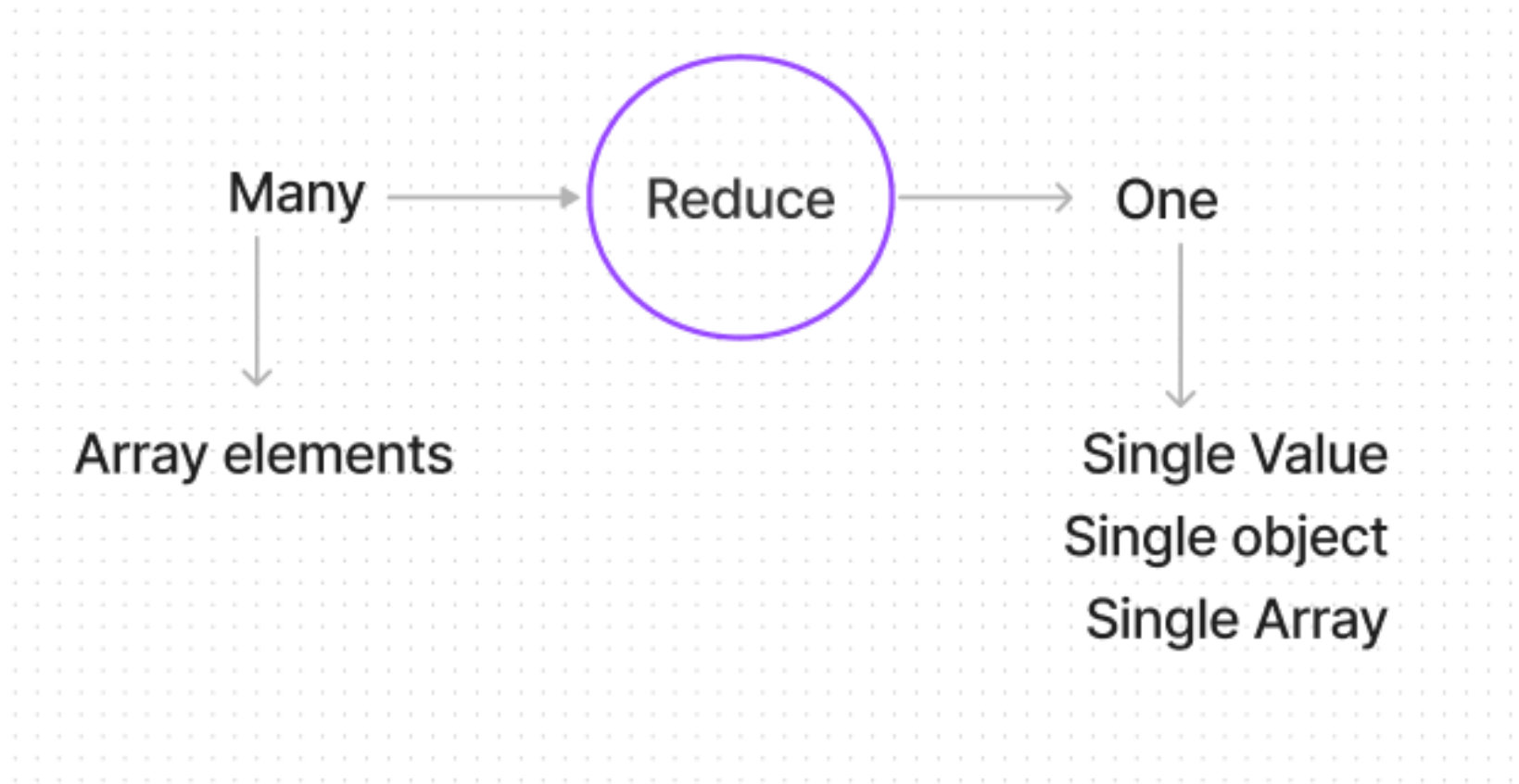


Reduction

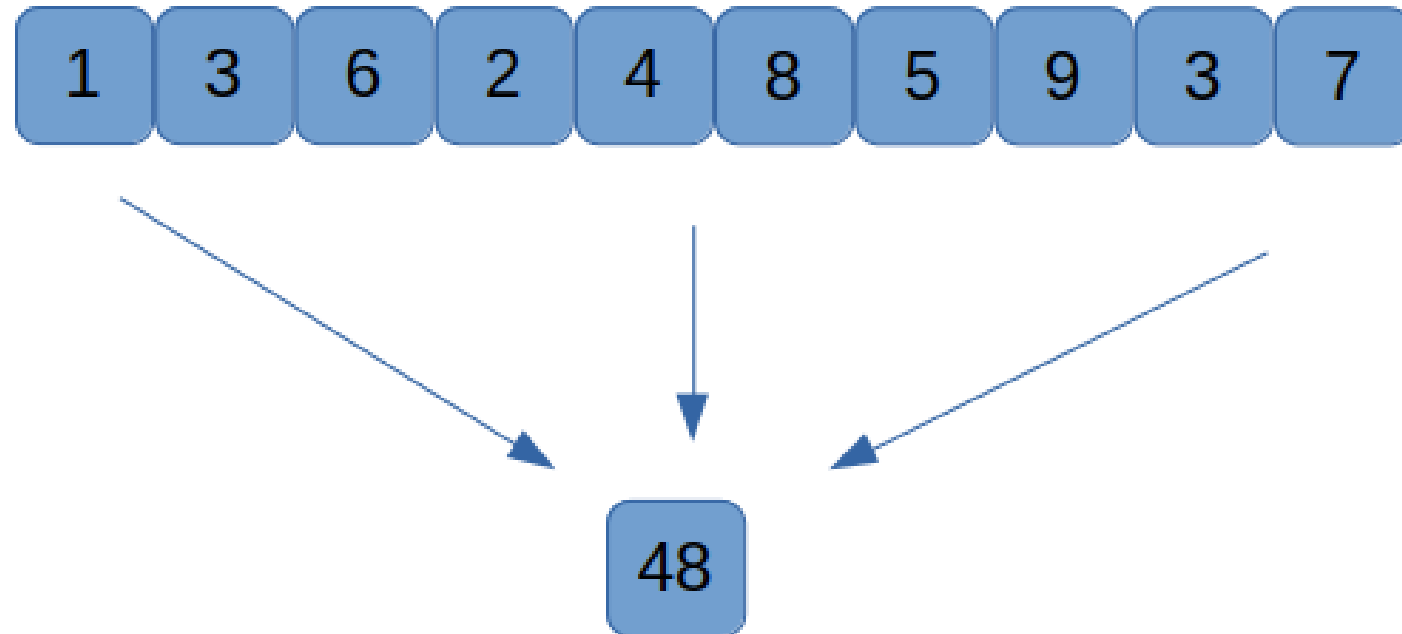
- **Example #5**

- 1 - Try to implement reduction in parallel using PyCUDA using one thread block
 - Extend it to use arbitrary size (i.e., multiple thread blocks)
- 2- Try to implement reduction in parallel using Numba CPU
- 3- Try to implement reduction in parallel using Numba GPU
- 4 - Compare execution times
 - Consider the Python reduce operator

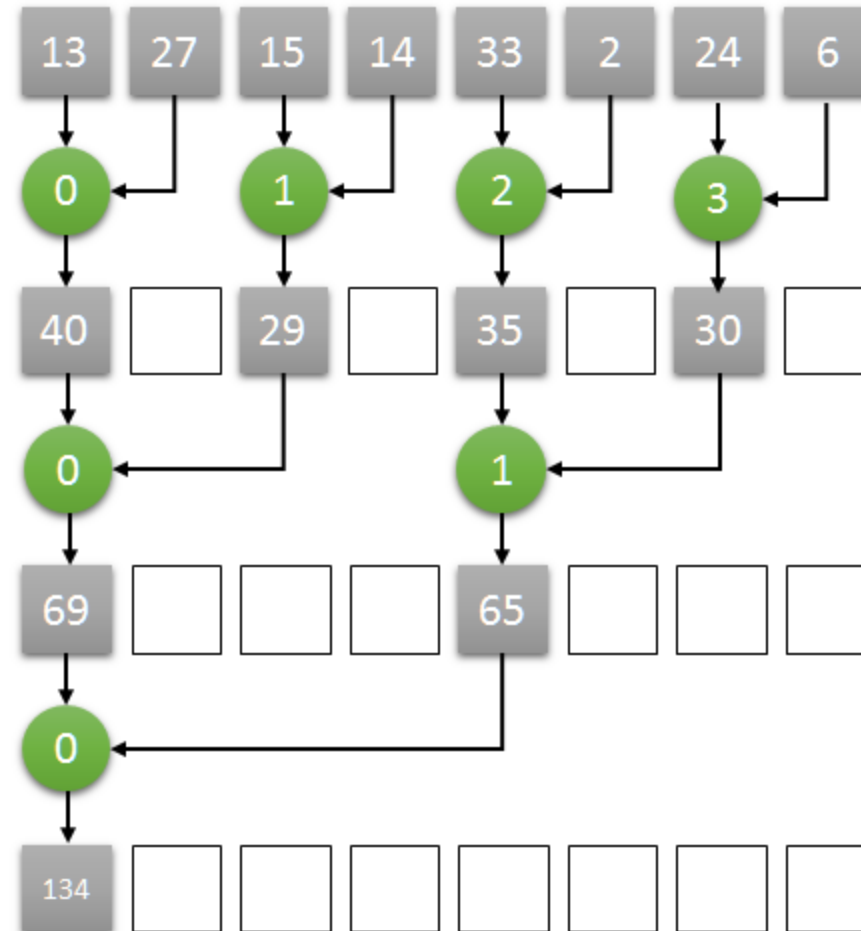
Reduction



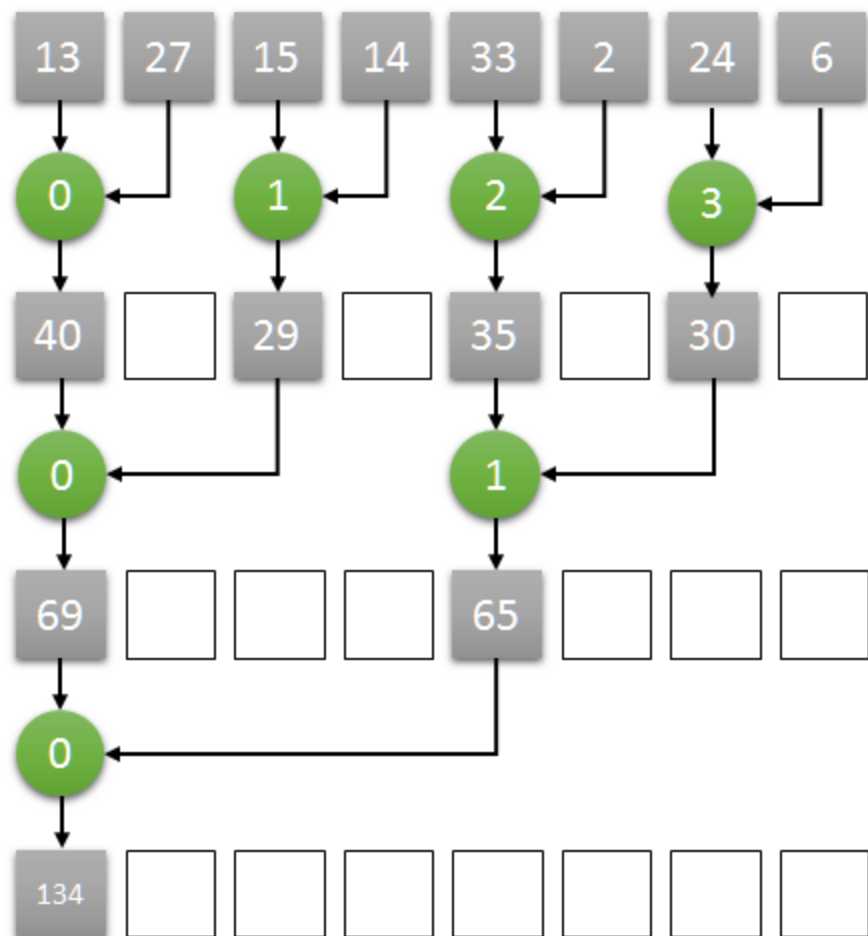
Reduction (addition)



Reduction (addition)



Reduction (addition)



level

1

$(tid \% 2 == 0) ==> a[tid] += a[tid+1]$

2

$(tid \% 4 == 0) ==> a[tid] += a[tid+2]$

3

$(tid \% 8 == 0) ==> a[tid] += a[tid+4]$

4

$(tid \% 16 == 0) ==> a[tid] += a[tid+8]$

$(tid \% (2^{\text{level}}) == 0) ==> a[tid] += a[tid+2^{(\text{level}-1)}]$

Reduction

- **Example #5**

- 1 - Try to implement reduction in parallel using PyCUDA using one thread block
 - Extend it to use arbitrary size (i.e., multiple thread blocks)
- 2- Try to implement reduction in parallel using Numba CPU
- 3- Try to implement reduction in parallel using Numba GPU
- 4 - Compare execution times
 - Consider the Python reduce operator

Reduction

- How to use shared memory in reduction?
- Compare its execution time to the sequential version and Python reduce operator



BASICS OF PERFORMANCE ANALYSIS AND PROFILING WITH NSIGHT COMPUTE PROFILING TOOL (NCU)

Outline

- Profiling Tool - What it is and does
- Nsight Compute Profiling Tool (ncu)
- Profiling Applications on Snellius

Profiling Tool

What it is and does

- Characterizes hardware **performance metrics**
- Detects **performance issues**
- Supports **analysis** workflows
- Enables **baseline comparisons** across GPU architectures for cross-platform development
- Shows how performance relates to **underlying hardware concepts**
- Helps users identify **potential bottlenecks** and **underutilization**
- Correlates **efficiency** and **performance** metrics down to individual **lines of code**

Profiling Tool

What it is and does

Performance:

- **How fast** a task is completed by the computer
 - Execution time
 - Throughput
 - Bandwidth
 - FLOPS

Efficiency:

- **How well** the computational resources are used to achieve a given performance
 - Resource utilization
 - % of peak performance

Profiling Tool

What it is and does

- Performance Analysis
- Characterizes your workflow in
 - **Memory-bound**
 - Memory throughput is high
 - **Compute-bound**
 - Compute throughput is high

Profiling Tool

What it is and does

GPU Performance:

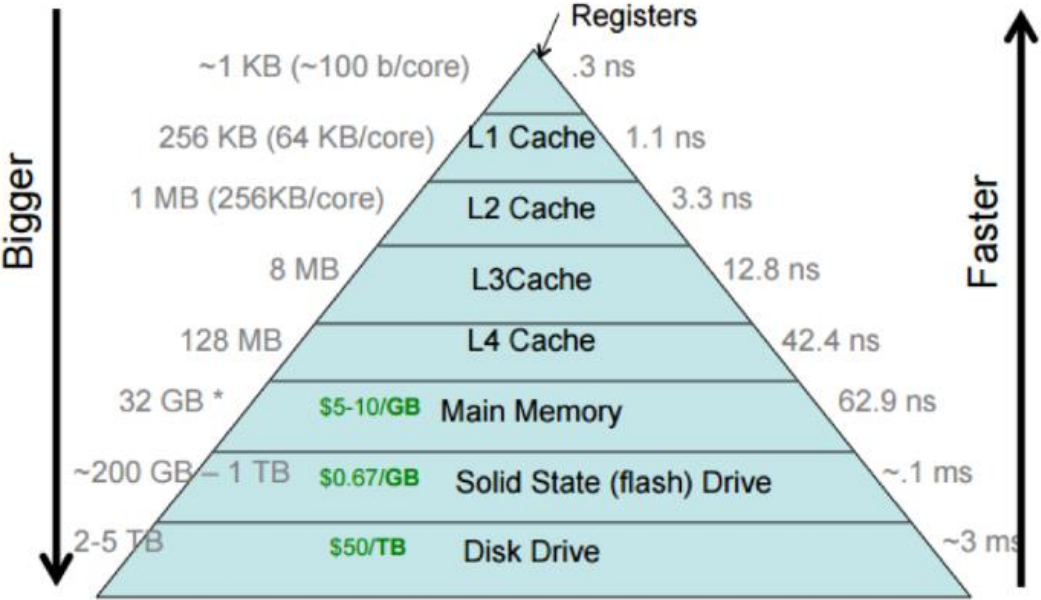
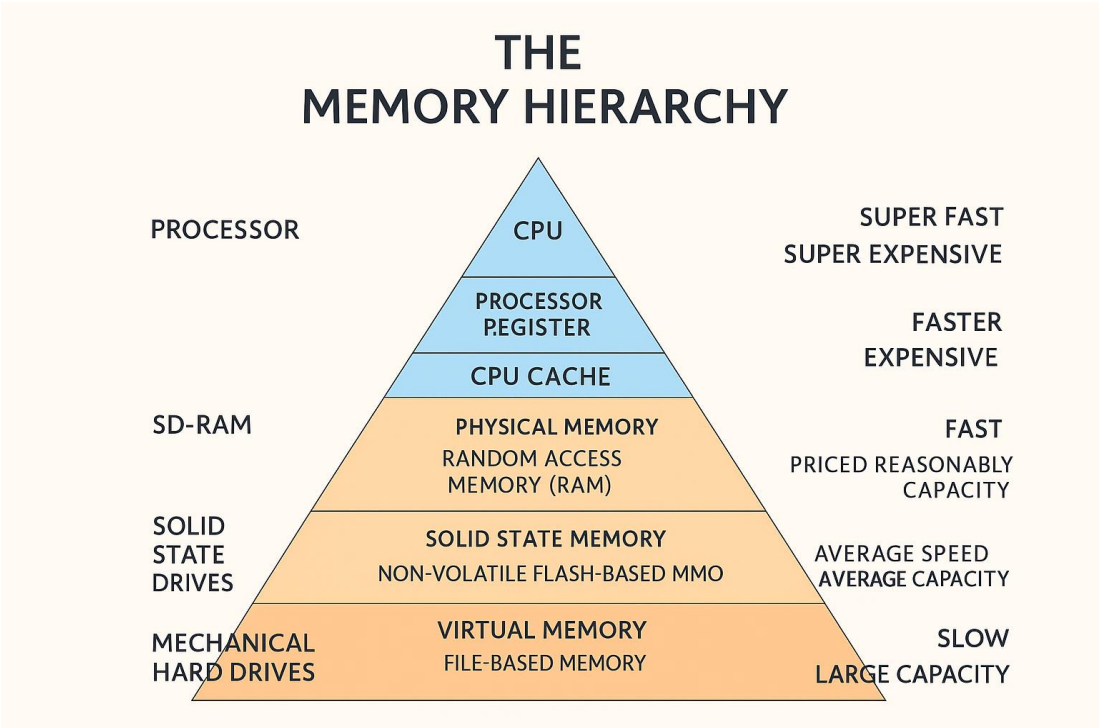
- **Unoptimized GPU programs are expensive** in terms of time, power and money
- Optimized GPU programs need to take advantage of several **execution units**
- High **trade-off between** development **time** and optimization **effort**

Think about:

- What is the purpose of using GPUs, and how is this code expected to evolve over time?
- What do I need in terms of performance? Fast execution times? Better throughput? Better occupancy of the GPU node?

Profiling Tool

Memory Hierarchy



Profiling Tool

Which tasks are best candidates for GPU?

✓ **Best** suited for GPU acceleration:

- High data parallelism
- Tasks that perform the same operations on many independent data elements.
- High computational intensity and low memory intensity
- Large amounts of computation per byte transferred.
- Examples include:
- Dense linear algebra
- Matrix–matrix multiplication
- Simulation kernels
- FFTs
- Monte Carlo methods
- Convolutions
- Physics simulations

✗ **Less** suitable for GPUs:

- Memory-heavy applications
- Workloads dominated by data movement rather than computation
- Frequent, small I/O operations
- Especially between host and device or network I/O.
- Irregular memory access patterns
- Unpredictable or scattered data access reduces GPU efficiency.
- Significant branching or divergence
- Heavy use of if–else, switch, or other divergent control flow.

Nsight Compute Profiling Tool (ncu)

What it is

- Interactive **kernel profiler** for CUDA applications
- Identifies **performance** and **efficiency** bottlenecks
- Analyses the device **memory usage**
- Translates metric values into **actionable** information
- It provides:
 - Detailed **performance metrics**
 - **API debugging**
- Can be used through **user interface (UI)** and **command line (CLI)** tool
 - (This course will cover the command line tool part)
- Differs from Nsight System (Nsys) profiler that provides **system-wide** profiling
- Ncu offers **kernel-based** profiling

Nsight Compute Profiling Tool (ncu)

Where to find

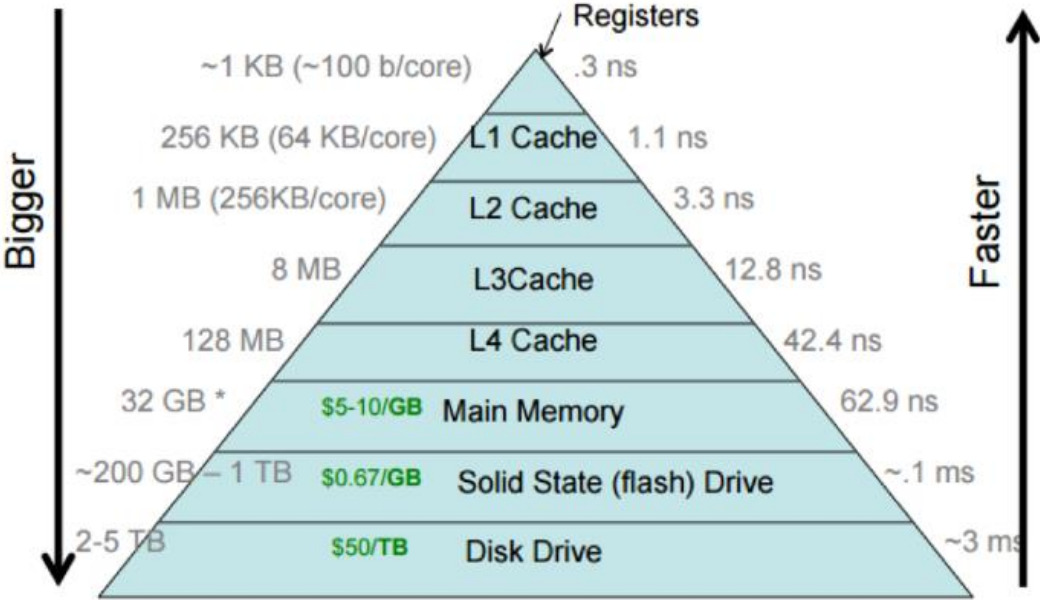
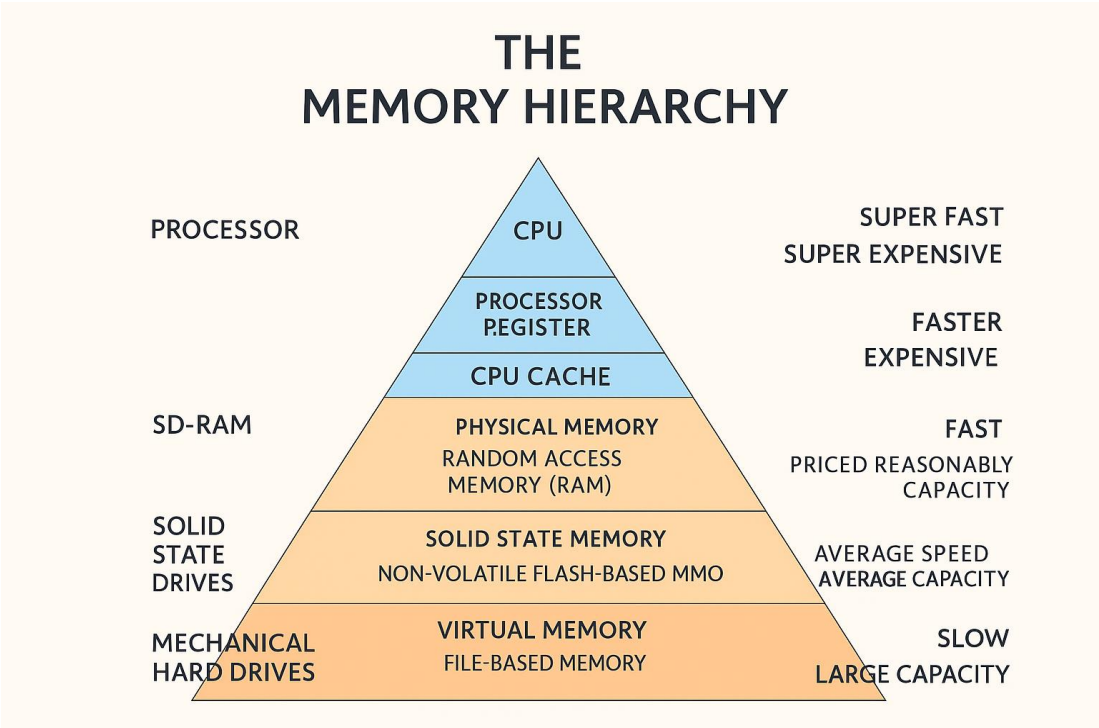
- Is available under the regular CUDA Toolkit installation
- On Snellius:

module load 2025

module load CUDA/12.8.0

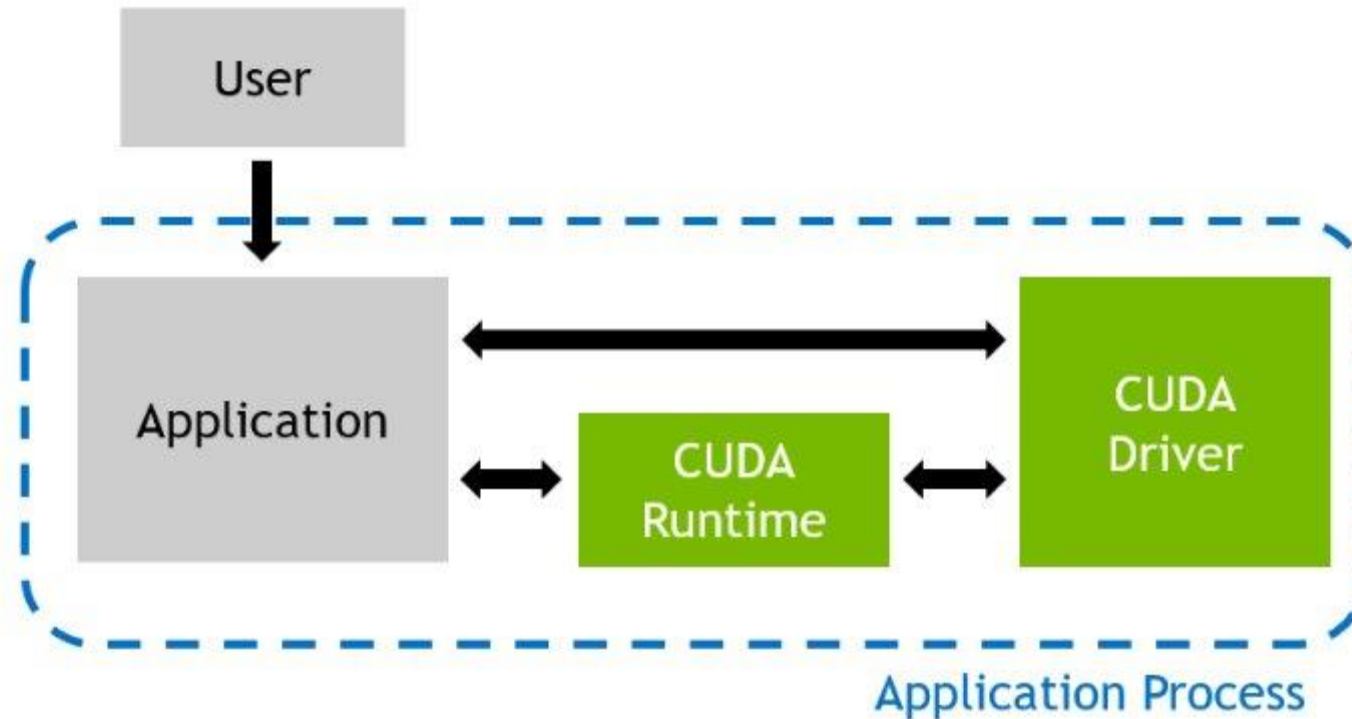
Profiling Tool

Memory Hierarchy



Nsight Compute Profiling Tool (ncu)

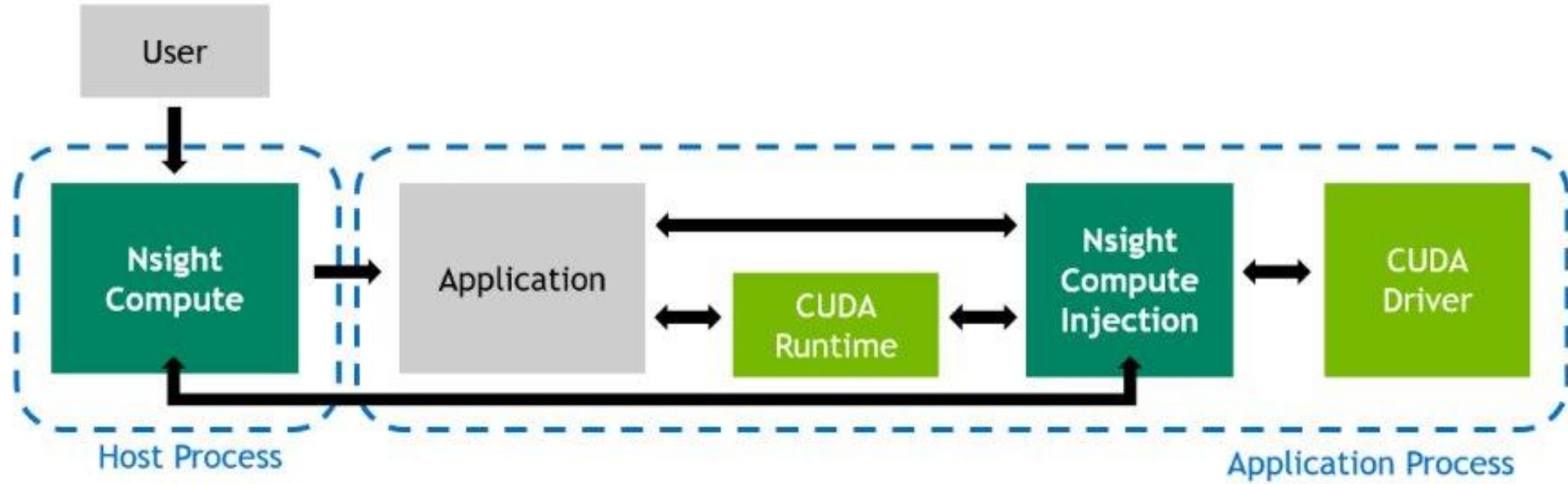
Profiling Applications



Regular Application Execution

Nsight Compute Profiling Tool (ncu)

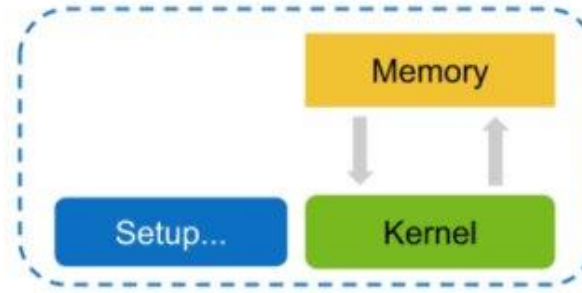
Profiling Applications



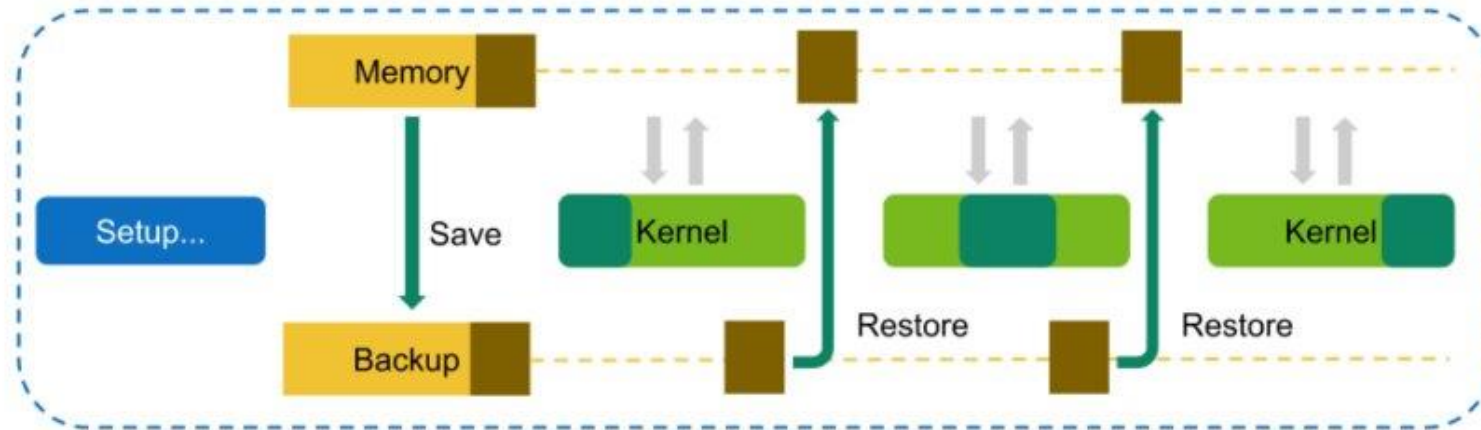
Profiled Application Execution

Nsight Compute Profiling Tool (ncu)

Metric Collection: Kernel Replay



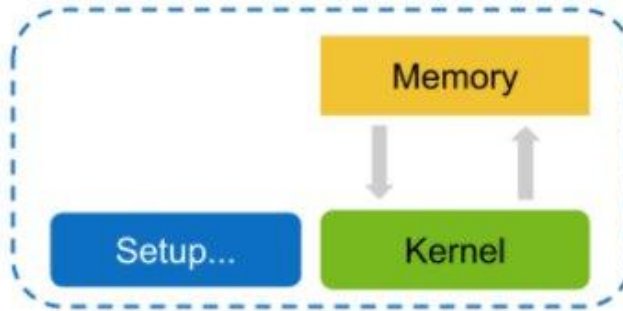
Regular Application Execution



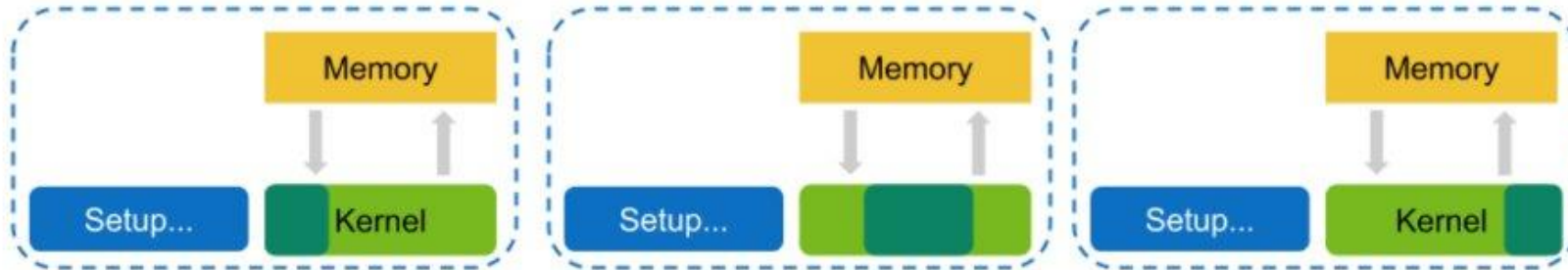
Execution with Kernel Replay. All memory is saved, and memory written by the kernel is restored in-between replay passes.

Nsight Compute Profiling Tool (ncu)

Metric Collection: Application Replay



Regular Application Execution



Execution with Application Replay. No memory is saved or restored, but the cost of running the application itself is duplicated.

Nsight Compute Profiling Tool (ncu)

Metric Collection: Sets and Sections

- Commonly used subset of metrics are grouped into pre-defined sets or sections
- Each set and section added adds also an overhead to the whole collection

Identifier

ComputeWorkloadAnalysis (Compute Workload Analysis)

InstructionStats (Instruction Statistics)

LaunchStats (Launch Statistics)

MemoryWorkloadAnalysis (Memory Workload Analysis)

NUMA Affinity (NumaAffinity)

Nvlink (Nvlink)

Nvlink_Tables (Nvlink_Tables)

Nvlink_Topology (Nvlink_Topology)

PM Sampling (PmSampling)

PM Sampling: Warp States (PmSampling_WarpStates)

SchedulerStats (Scheduler Statistics)

SourceCounters (Source Counters)

SpeedOfLight (GPU Speed Of Light Throughput)

WarpStateStats (Warp State Statistics)

Nsight Compute Profiling Tool (ncu)

Metric Collection: Default

- Default collection:
 - High-level utilization
 - Static Launch
 - Occupancy

Identifier

ComputeWorkloadAnalysis (Compute Workload Analysis)

LaunchStats (Launch Statistics)

MemoryWorkloadAnalysis (Memory Workload Analysis)

SpeedOfLight (GPU Speed Of Light Throughput)

WarpStateStats (Warp State Statistics)

Occupancy (Occupancy)

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

- **Basic collection:**
 - `ncu python 3-vector-add-v0.py`

```
cliciad@gcn30 basic_examples]$ ncu -o output.txt python 1-vector-add-v0.py
```

```
==PROF== Connected to process 466370 (/gpfs/admin/_hpc/sw/arch/INTEL-  
AVX512/RHEL9/EB_production/2024/software/Python/3.12.3-GCCcore-13.3.0/bin/python3.12)
```

```
==PROF== Profiling "addition" - 0: 0%....50%....100% - 10 passes
```

```
Elapsed on GPU with PyCuda (sec): 1.3678835449218751
```

```
-----  
Elapsed time using CPU sequential for-loop (sec): 4.398094415664673
```

```
-----  
Validation: there are 0 different element(s)!
```

```
-----  
==PROF== Disconnected from process 1858485
```

```
[466370] python3.13@127.0.0.1
```

```
addition (19532, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.0
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

- **Basic collection:**
 - `ncu python 3-vector-add-v0.py`

```
cliciad@gcn30 basic_examples]$ ncu -o output.txt python 1-vector-add-v0.py
```

```
==PROF== Connected to process 466370 (/gpfs/admin/_hpc/sw/arch/INTEL-  
AVX512/RHEL9/EB_production/2024/software/Python/3.12.3-GCCcore-13.3.0/bin/python3.12)
```

```
==PROF== Profiling "addition" - 0: 0%....50%....100% - 10 passes
```

```
Elapsed on GPU with PyCuda (sec): 1.3678835449218751
```

```
-----  
Elapsed time using CPU sequential for-loop (sec): 4.398094415664673
```

```
-----  
Validation: there are 0 different element(s)!
```

```
-----  
==PROF== Disconnected from process 1858485
```

```
[466370] python3.13@127.0.0.1
```

```
addition (19532, 1, 1)x(512, 1, 1), Context 1, Stream 7, Device 0, CC 8.0
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

Section: GPU Speed Of Light Throughput		
Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	1.20
SM Frequency	Ghz	1.08
Elapsed Cycles	cycle	4,428
Memory Throughput	%	12.76
DRAM Throughput	%	12.76
Duration	us	4.10
L1/TEX Cache Throughput	%	7.81
L2 Cache Throughput	%	14.60
SM Active Cycles	cycle	2,111.67
Compute (SM) Throughput	%	3.24

Section: GPU and Memory Workload Distribution		
Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	627.10
Total DRAM Elapsed Cycles	cycle	198,656
Average L1 Active Cycles	cycle	2,111.91
Total L1 Elapsed Cycles	cycle	499,844
Average L2 Active Cycles	cycle	1,814.34
Total L2 Elapsed Cycles	cycle	344,160
Average SM Active Cycles	cycle	2,111.91
Total SM Elapsed Cycles	cycle	499,844
Average SMSP Active Cycles	cycle	2,204.97
Total SMSP Elapsed Cycles	cycle	1,999,376

Section: Launch Statistics		
Metric Name	Metric Unit	Metric Value
Block Size		512
Function Cache Configuration	CachePreference	None
Grid Size		196
Registers Per Thread	register/thread	16
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	108
Stack Size		1,024
Threads	thread	100,352
# TPCs		54
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		0.45

Section: Occupancy		
Metric Name	Metric Unit	Metric Value
Block Limit SM	block	32
Block Limit Registers	block	8
Block Limit Shared Mem	block	16
Block Limit Warps	block	4
Theoretical Active Warps per SM	warp	64
Theoretical Occupancy	%	100
Achieved Occupancy	%	42.42
Achieved Active Warps Per SM	warp	27.15

OPT Est. Local Speedup: 57.58%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (42.4%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide (<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#occupancy>) for more details on optimizing occupancy.

OPT If you execute `__syncthreads()` to synchronize the threads of a block, it is recommended to have at least two blocks per multiprocessor (compared to the currently executed 1.8 blocks) This way, blocks that aren't waiting for `__syncthreads()` can keep the hardware busy.

OPT This kernel grid is too small to fill the available resources on this device, resulting in only 0.5 full waves across all SMs. Look at Launch Statistics for more details.

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000)

```
OPT  Est. Speedup: 5.41%
      One or more SMs have a much lower number of active cycles than the average number of active cycles. Maximum
      instance value is 11.86% above the average, while the minimum instance value is 27.51% below the average.
-----
OPT  Est. Speedup: 6.235%
      One or more SMSPs have a much lower number of active cycles than the average number of active cycles. Maximum
      instance value is 13.09% above the average, while the minimum instance value is 28.62% below the average.
-----
OPT  Est. Speedup: 5.41%
      One or more L1 Slices have a much lower number of active cycles than the average number of active cycles.
      Maximum instance value is 11.86% above the average, while the minimum instance value is 27.51% below the
      average.
-----
OPT  Est. Speedup: 12.21%
      One or more L2 Slices have a much higher number of active cycles than the average number of active cycles.
      Maximum instance value is 28.96% above the average, while the minimum instance value is 9.33% below the
      average.
```

Nsight Compute Profiling Tool (ncu)

Example 1: vector-add-v2.py (N = 100,000,000)

Section: GPU Speed Of Light Throughput

Metric Name	Metric Unit	Metric Value
DRAM Frequency	Ghz	1.21
SM Frequency	Ghz	1.09
Elapsed Cycles	cycle	951,181
Memory Throughput	%	88.28
DRAM Throughput	%	88.28
Duration	us	868.70
L1/TEX Cache Throughput	%	16.20
L2 Cache Throughput	%	69.27
SM Active Cycles	cycle	944,526.70
Compute (SM) Throughput	%	12.21

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	32
Block Limit Registers	block	8
Block Limit Shared Mem	block	16
Block Limit Warps	block	4
Theoretical Active Warps per SM	warp	64
Theoretical Occupancy	%	100
Achieved Occupancy	%	85.51
Achieved Active Warps Per SM	warp	54.73

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		512
Function Cache Configuration	CachePreferNone	
Grid Size		195,313
Registers Per Thread	register/thread	16
Shared Memory Configuration Size	Kbyte	16.38
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	108
Stack Size		1,024
Threads	thread	100,000,256
# TPCs		54
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		452.11

OPT Est. Local Speedup: 14.49%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (85.5%) can be the result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalances can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Best Practices Guide (<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#occupancy>) for more details on optimizing occupancy.

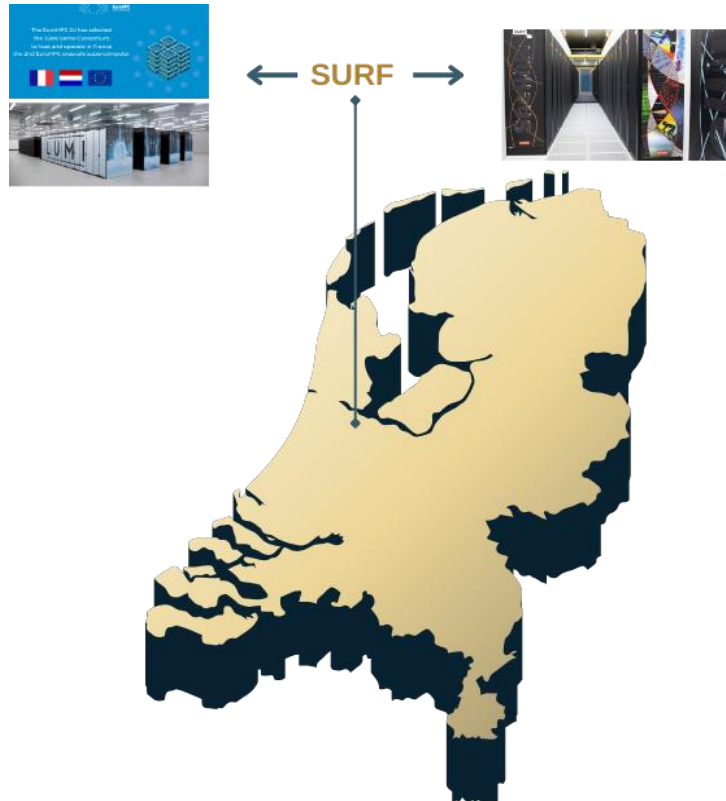
INF This workload is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing DRAM in the Memory Workload Analysis section.

Independent numerical work is a good starting point

Promising workload	Potential obstacle
Same operation over many elements	Small arrays and launch overhead
Dense linear algebra with reusable data	Unnecessary copies between operations
Stencils and local neighborhoods	Memory traffic
Irregular or sequential algorithms	Dependencies, divergence, and poor locality

Access routes and planned systems

Current



Future

- **Alice Recoque Exascale System (France - 2026)**
The Alice Recoque supercomputer will be hosted and operated by the Jules Verne consortium, led by France through [GENCI](#) and [CEA](#), with the participation of the Netherlands through [SURF](#).
- **AI Factory Groningen - 2027**
consortium partners SURF, TNO and Samenwerking Noord, has taken the initiative for this facility and is working on its implementation, aiming to have it fully operational by 2027.

Thank You

Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and Germany, Bulgaria, Austria, Croatia, Cyprus, Czech Republic, Denmark, Estonia, Finland, Greece, Hungary, Ireland, Italy, Lithuania, Latvia, Poland, Portugal, Romania, Slovenia, Spain, Sweden, France, Netherlands, Belgium, Luxembourg, Slovakia, Norway, Türkiye, Republic of North Macedonia, Iceland, Montenegro, Serbia under grant agreement No 101101903.